

Intelligent Compliance Management System (ICMS)



MUHAMMAD ALI BUKHARI
01-135222-096

MUHAMMAD HAMMAD CHAGHTAI
01-135222-014

Supervisor: Dr. Faisal Bashir

Bachelor of Science in Information Technology

Department of Computer Science
Bahria University, Islamabad

April 2026

Certificate

We accept the work contained in the report title “**Intelligent Compliance Management System (ICMS)**” written by **Mr. Muhammad Ali Bukhari** and **Mr. Muhammad Hammad Chaghtai** as a confirmation to the required standard for the partial fulfillment of the degree of Bachelor of Science in Information Technology.

Approved by ...:

Supervisor: Dr. Faisal Bashir (Dean Engineering Sciences)

Internal Examiner: Name of the Internal Examiner (Title)

External Examiner: Name of the External Examiner (Title)

Project Coordinator: Dr. Kabeer Ahmed Bhatti (FYP Coordinator, IT Department)

Head of the Department: Dr. Muhammad Khurram Ehsan (Head of Department, Computer Science)

April , 2026

Abstract

Manual security auditing and static compliance mapping create temporal blind spots, leaving enterprise infrastructure vulnerable to configuration drift between assessment cycles. Traditional Governance, Risk, and Compliance (GRC) processes often remain disconnected from actual endpoint telemetry, resulting in high administrative overhead and inaccurate security postures. This project presents the Intelligent Compliance Management System (ICMS), a centralized orchestration platform designed to automate continuous controls monitoring. Originating as an industrial collaborative project, the platform integrates directly with the Wazuh Security Information and Event Management (SIEM) system to ingest unstructured Security Configuration Assessment (SCA) logs. By implementing a deterministic mapping algorithm, the ICMS translates raw technical events into verifiable regulatory controls, enabling continuous compliance tracking against standards such as ISO/IEC 27001.

The system is engineered using an N-tier architecture. A React Single Page Application (SPA) serves as the presentation layer, communicating with a Django REST Framework API. To prevent main-thread blocking during high-volume SIEM polling and comprehensive PDF report generation, intensive I/O operations are offloaded to an asynchronous Celery and Redis worker tier. Data persistence is managed via PostgreSQL, utilizing a hybrid schema that combines rigid relational constraints for Role-Based Access Control (RBAC) with dynamic JSON fields for variable telemetry storage. Platform security is enforced through stateless JSON Web Tokens (JWT) and Time-based One-Time Password (TOTP) two-factor authentication.

Comprehensive functional and non-functional testing confirmed the platform's operational viability. The asynchronous tier successfully processed high-density endpoint synchronizations while maintaining an average API latency of under 150ms, and the mapping engine maintained strict referential integrity against collision attempts. The ICMS demonstrates that replacing point-in-time manual audits with continuous, mathematically verifiable SIEM integration establishes a deterministic compliance posture and significantly reduces organizational audit fatigue.

Acknowledgments

In the name of Allah, the Most Gracious, the Most Merciful. All praise is due to Allah, the Almighty, for granting the strength and guidance required to complete this work.

We express our deepest gratitude to our supervisor, Dr. Faisal Bashir, for his mentorship, technical expertise, and continuous encouragement throughout the development of this project. His insights into systems architecture and cybersecurity were fundamental to the successful execution of this project. We also extend our sincere appreciation to the dedicated lab staff who assisted us during our practical work, and to the diverse faculty members whose teaching and companionship shaped our understanding over the years. We are profoundly thankful to the Department of Computer Science and Bahria University, not only for providing the necessary resources for our project but for being a place of invaluable life experiences and holistic learning.

We owe an immense debt of gratitude to our parents, families, and friends for their prayers, unconditional support, and patience throughout this intensive process. We also take a moment to thank one another; navigating the complexities of this project side-by-side and seeing it through to accomplishment has been a remarkable journey of genuine teamwork and shared dedication.

Finally, on a deeply personal note, I (Muhammad Ali Bukhari) would like to dedicate a special remembrance to my beautiful pet bird, Kiroo. Though he is no longer with me today, his presence meant the world to me, and his memory provided quiet comfort and joy during the long hours of this journey.

Muhammad Ali Bukhari

Muhammad Hammad Chaghtai

Contents

Certificate	i
Abstract	ii
Acknowledgments	iii
List of Figures	x
List of Tables	xii
List of Abbreviations	xiv
1 Introduction	1
1.1 Project Background	1
1.1.1 The Temporal Blind Spot of Audits	1
1.1.2 The Shift to Continuous Monitoring	1
1.2 Problem Description	2
1.2.1 Telemetry Correlation Delays	2
1.2.2 Algorithmic Failures in Legacy Systems	2
1.3 Project Objectives	3
1.4 Methodology	4
1.4.1 Agile Development Lifecycle	4
1.4.2 Sandbox Testing and Evaluation	4
1.5 Project Scope	5
1.5.1 Compliance Framework Boundaries	5

1.5.2	Technical Implementation Boundaries	5
1.6	Feasibility Study	6
1.6.1	Technical and Economic Feasibility	6
1.6.2	Operational Feasibility	6
1.7	Areas of Application	6
1.7.1	Financial Technology (FinTech)	6
1.7.2	Healthcare Sector	7
1.8	Report Organization	7
2	Literature Review	8
2.1	Comprehensive Literature Review	8
2.1.1	Evolution of Governance, Risk, and Compliance (GRC) Platforms	8
2.1.2	Security Information and Event Management (SIEM) Integrations	8
2.1.3	Performance Bottlenecks in Telemetry Aggregation	9
2.1.4	Modern Access Control Paradigms	10
2.1.5	Containerization and Defense-in-Depth Topology	11
2.1.6	Cognitive Load and HCI in Security Dashboards	12
2.2	Analysis of Existing Systems	12
2.2.1	Manual Spreadsheet Auditing	12
2.2.2	Vanta (Commercial SaaS)	12
2.2.3	Drata (Commercial SaaS)	13
2.2.4	AuditBoard (Enterprise GRC)	13
2.3	Comparative Analysis	14
2.4	Summary of Competitive Advantages	14

3	Requirement Specification	16
3.1	Existing System	16
3.2	Proposed System	16
3.2.1	Technology Stack Selection	17
3.2.2	High-Level Architecture	18
3.2.3	Dockerized Infrastructure and Network Isolation	19
3.3	Requirements Specification	20
3.4	Functional Requirements	21
3.5	Non-Functional Requirements	21
3.6	User Requirements	22
3.7	System Models and Components	22
3.7.1	Context Diagram	22
3.7.2	Data Flow Diagram	23
3.8	Use Cases	24
3.8.1	Use Case Diagram	24
3.8.2	Descriptive Use Cases	25
4	Design	32
4.1	System Architecture	32
4.1.1	Client and Presentation Tier	32
4.1.2	Application and Middleware Tier	32
4.1.3	Asynchronous Processing Tier	32
4.1.4	Data Persistence Tier	33
4.2	Design Constraints	33

4.2.1	Technical Constraints	33
4.2.2	Infrastructure and Operational Constraints	34
4.3	Design Methodology	34
4.4	High Level Design	35
4.4.1	Component Diagram	35
4.4.2	Deployment Diagram	36
4.4.3	System Activity Diagrams	37
4.4.4	System Sequence Diagrams	43
4.5	Low Level Design	46
4.5.1	UML Class Diagram	46
4.6	Database Design	47
4.6.1	Physical Data Model (Core ERD)	47
4.6.2	Extended Entity Relationship Diagram (EERD)	48
4.7	Graphical User Interface (GUI) Design	49
4.7.1	Authentication and Identity Management	49
4.7.2	Superadmin Configuration Portal	50
4.7.3	Admin and Auditor Workspaces	51
4.7.4	Automated PDF Audit Artifacts	53
4.8	External Interfaces	54
4.8.1	Wazuh SIEM REST API	54
4.8.2	SMTP Notification Gateway	54
4.8.3	Outbound Webhook Integration	55
4.8.4	Interface Specification Matrix	55

5	Implementation	56
5.1	System Architecture	56
5.1.1	Presentation Layer Implementation (Frontend)	58
5.1.2	Middleware and API Gateway (Backend)	58
5.1.3	Asynchronous Processing Engine	59
5.1.4	Data Persistence Layer	60
5.1.5	Architectural Technology Stack	60
5.2	System Internal Components	61
5.2.1	Identity and Access Management (IAM) Module	62
5.2.2	Governance and Compliance Data Engine	62
5.2.3	SIEM Telemetry and Synchronization Module	63
5.2.4	Automated Audit and Reporting Module	63
5.2.5	Frontend State and Visualization Layer	64
5.3	Tools and Technologies	65
6	System Testing and Evaluation	67
6.1	Critical Appraisal and Limitations	67
6.2	Testing Environment and Methodology	68
6.3	Functional System Test Cases	68
6.4	Non-Functional System Testing	82
6.4.1	Performance and Load Testing	83
6.4.2	Security and Penetration Testing	83
6.4.3	Usability and Accessibility Testing	84
6.4.4	Compatibility and Deployment Testing	85

7	Conclusions	87
A	User Manual	89
A.1	Purpose	89
A.2	System Requirements	89
A.3	System Access	89
A.4	Main User Steps	90
A.5	Logout	91
A.6	Support	92
A.7	Note	92
	Bibliography	93

List of Figures

3.1	Refined N-Tier System Architecture and Protocol Flow	19
3.2	Docker Service Mesh: Port Exposure and Health-Check Dependencies . .	20
3.3	Context Diagram: High-Level System Interactions	23
3.4	Level-1 Data Flow Diagram detailing internal compliance processes. . . .	24
3.5	System Use Case Diagram	25
4.1	Tier-wise System Architecture of the ICMS Platform	33
4.2	Conceptual View: UML Component Diagram	36
4.3	Physical View: UML Deployment Diagram	36
4.4	Activity Diagram: Authentication and IAM Routing	38
4.5	Activity Diagram: Parallel Asynchronous Telemetry Ingestion	39
4.6	Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority	40
4.7	Activity Diagram: Automated PDF Audit Report Generation	41
4.8	Activity Diagram: Framework Rule Mapping and API Validation	42
4.9	Activity Diagram: Real-Time Alerting and Notification Fan-out	43
4.10	Sequence Diagram: Authentication Flow with UML Alt Fragment	44
4.11	Sequence Diagram: Background Telemetry Synchronization with UML Loop	45
4.12	Sequence Diagram: Framework Rule Mapping	45
4.13	Sequence Diagram: Report Generation with Alternative Data States	46
4.14	UML Class Diagram: Core Object-Relational Models	47
4.15	Physical Database Schema: Core Relational Data Model with Exact Data Types	48

4.16	Extended Entity Relationship Diagram (EERD) with Constraints and Attributes	49
4.17	GUI Design: Secure User Authentication Portal	50
4.18	GUI Design: Superadmin Global Compliance Dashboard	50
4.19	GUI Design: Framework and Wazuh Rule Mapping Interface	51
4.20	GUI Design: Monitored Endpoints and Telemetry Synchronization Status	51
4.21	GUI Design: System Admin Operational Dashboard	52
4.22	GUI Design: Security Auditor Read-Only Compliance View	52
4.23	GUI Design: Auditor Report Generation Modal	52
4.24	System Output: Automated PDF Report - Cover and Executive Summary	53
4.25	System Output: Automated PDF Report - Aggregated Framework Control Matrix	53
4.26	System Output: Automated PDF Report - Technical Wazuh Telemetry Evidence	54
5.1	Implemented N-Tier System Architecture	57
5.2	Implemented Service Topology and Container Networking	57

List of Tables

2.1	Comparative Feature Matrix: Analyzing GRC Platforms	14
3.1	Use Case - Authentication	26
3.2	Use Case - Manage Frameworks	27
3.3	Use Case - Map Controls	28
3.4	Use Case - Sync Telemetry	29
3.5	Use Case - View Dashboard	30
3.6	Use Case - Export Audit Reports	31
4.1	External Interface Technical Specifications	55
5.1	Architectural Technology Stack and Implementation Roles	61
5.2	Summary of Tools, Technologies, and Development Stack	65
6.1	TC-01: Valid Superadmin Authentication (Positive)	69
6.2	TC-02: Invalid Credential Rejection (Negative)	70
6.3	TC-03: Global Theme Toggling — Light/Dark Mode (Positive)	71
6.4	TC-04: UI RBAC View Segregation (Positive)	72
6.5	TC-05: Unauthorized API Write Attempt (Negative)	73
6.6	TC-06: Deterministic Framework Mapping (Positive)	74
6.7	TC-07: Duplicate Mapping Collision (Negative)	75
6.8	TC-08: Wazuh API Handshake and Token Retrieval (Positive)	76
6.9	TC-09: Wazuh Connection Timeout Exception (Negative)	77

6.10	TC-10: Asynchronous Telemetry Polling via Celery Beat (Positive) . . .	78
6.11	TC-11: Unstructured JSON Log Ingestion (Positive)	79
6.12	TC-12: Dynamic Compliance Score Recalculation (Positive)	80
6.13	TC-13: Comprehensive PDF Audit Report Generation (Positive)	81
6.14	TC-14: Asynchronous SMTP Alert Dispatch (Positive)	82
6.15	Performance Test Results	83
6.16	Security Test Results	84
6.17	Usability Test Task Completion Results	85
6.18	Compatibility and Deployment Test Results	86

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CCM	Continuous Controls Monitoring
CI/CD	Continuous Integration and Continuous Deployment
CPU	Central Processing Unit
DOM	Document Object Model
DRF	Django REST Framework
ERD	Entity-Relationship Diagram
FedRAMP	Federal Risk and Authorization Management Program
FISMA	Federal Information Security Modernization Act
FK	Foreign Key
GRC	Governance, Risk, and Compliance
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HIPAA	Health Insurance Portability and Accountability Act
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I/O	Input / Output
ICMS	Intelligent Compliance Management System
IDE	Integrated Development Environment
IEC	International Electrotechnical Commission
ISMS	Information Security Management System
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions

NIST	National Institute of Standards and Technology
ORM	Object-Relational Mapping
OS	Operating System
PDF	Portable Document Format
PHI	Protected Health Information
PK	Primary Key
QA	Quality Assurance
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SCA	Security Configuration Assessment
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VPN	Virtual Private Network
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Project Background

1.1.1 The Temporal Blind Spot of Audits

Historical security auditing methodologies rely on manual evidence collection and point-in-time assessments. Organizations compile spreadsheets and static configuration snapshots to satisfy regulatory requirements. This approach creates temporal blind spots because the security posture remains unverified between audit cycles. Configuration drift occurs frequently after successful audits, leaving infrastructure vulnerable to exploitation despite maintaining formal compliance status.

1.1.2 The Shift to Continuous Monitoring

The cybersecurity industry addresses these vulnerabilities by transitioning toward Continuous Controls Monitoring (CCM). This transition replaces manual sampling with automated validation of security controls. CCM platforms integrate with the technical environment to evaluate configurations, access logs, and system integrity continuously. This automation ensures compliance reflects the current state of the environment rather than a historical record. Modern regulatory mandates codify the requirement for continuous visibility. NIST Special Publication 800-53 Revision 5 requires the transition from static control assessments to ongoing monitoring to validate system integrity [8]. Similarly, the ISO/IEC 27001:2022 standard requires organizations to establish an Information Security Management System (ISMS) that identifies and treats risks through continuous evaluation [2]. These frameworks require technological solutions capable of mapping raw system data to governance objectives. Security Information and Event Management (SIEM) systems provide the data foundation for this automated architecture. SIEM engines ingest raw telemetry from across the enterprise, translating system events into verifiable compliance states. By establishing deterministic baselines, organizations

move from reactive reporting to data-driven security management. This project evaluates the implementation of a system that bridges the gap between technical SIEM telemetry and regulatory control satisfaction.

1.2 Problem Description

1.2.1 Telemetry Correlation Delays

Translating large volumes of endpoint telemetry into regulatory frameworks presents an operational limitation. SIEM platforms generate technical alerts, such as SSH authentication failures or unauthorized file modifications, which require correlation against ISO 27001 or SOC 2 control matrices. Manual correlation processes introduce delays between event detection and compliance validation, creating risk windows where system vulnerabilities remain unaddressed. Concurrent telemetry sources produce contradictory states, requiring deterministic resolution mechanisms that manual workflows cannot provide.

1.2.2 Algorithmic Failures in Legacy Systems

Calculating compliance metrics across distributed fleets exposes data aggregation limitations in traditional GRC platforms. Standard relational database implementations suffer from the N+1 query problem, where the application executes sequential retrieval loops for individual security controls. This architectural flaw imposes a linear $O(N)$ algorithmic complexity on data retrieval, causing geometric performance decay as monitored agents increase [11]. The resulting latency prevents organizations from achieving continuous monitoring, because dashboard rendering times exceed thresholds for operational visibility. The algorithmic limitations of legacy GRC systems stem from their inability to process large datasets within acceptable temporal bounds. When an organization attempts to map raw telemetry events to multiple regulatory frameworks, application-level iteration induces memory bandwidth exhaustion. By fetching full relational objects through unoptimized query loops, these systems saturate computational resources, leading to timeouts and data inconsistency. This performance decay prevents the transition from point-in-time auditing to continuous monitoring, because system latency outpaces telemetry ingestion.

Legacy GRC interfaces increase cognitive load on security analysts. Rigid HTML table-heavy dashboards and unoptimized color schemes lead to rapid desensitization, known as

alert fatigue. Chronic overstimulation from unfiltered telemetry streams causes operators to ignore or misclassify high-severity threats [10]. This Human-Computer Interaction (HCI) shortcoming degrades incident response times and increases organizational risk. High-density monochromatic data grids lack semantic visual hierarchy, forcing operators to filter noise from critical signals manually. This structural rigidity induces alert fatigue, where the analyst's sensory threshold increases until critical exploits are overlooked. The absence of adaptive layout modules transforms these tools into sources of risk during security incidents.

1.3 Project Objectives

The development of the ICMS platform addresses the identified technical and operational gaps through four engineering objectives:

- **Telemetry Automation:** Automate the ingestion and mapping of Wazuh SIEM telemetry to ISO 27001 and SOC 2 controls, utilizing a deterministic Worst-Case Priority state-mutation algorithm to resolve conflicting evaluations. By polling the Wazuh Manager's Security Configuration Assessment (SCA) module, the platform retrieves check results and correlates them against a pre-seeded compliance matrix. This methodology guarantees the compliance dashboard reflects the most conservative security posture, minimizing false-positive compliance reports.
- **Performance Optimization:** Eliminate the $N+1$ database querying bottleneck by engineering $O(1)$ set-based aggregations through the Django ORM. The platform utilizes query annotation and conditional SQL filtering to constrain dashboard latency below 200 milliseconds. The system projects complex SQL constraints and conditional aggregations directly onto the relational layer instead of executing sequential retrieval loops. By computing fleet-wide compliance scores internally, the system reduces data transfer overhead and memory pressure on the application server.
- **Security Infrastructure:** Enforce strict infrastructure and session security through a Dockerized virtual bridge network that isolates the PostgreSQL data tier, combined with custom Django REST Framework Role-Based Access Control and stateless JWT rotation. The platform isolates the database within a private virtual network, ensuring the persistence tier remains unreachable from external networks. This physical isolation pairs with an Identity and Access Management (IAM) layer that utilizes stateless JSON Web Token (JWT) rotation and custom permission classes.

- **UI/UX Engineering:** Mitigate analyst alert fatigue by architecting a modern frontend that replaces rigid HTML tables with flexible layouts. The interface allows data rows to wrap and scale dynamically based on viewport size. The platform uses elevation through depth illusions and semantic color scaling to distinguish interactive security cards from static background telemetry, structuring raw data for decision support.

1.4 Methodology

1.4.1 Agile Development Lifecycle

The development of the ICMS platform adopted an Agile software development lifecycle to facilitate iterative progression and requirement adaptation. This engineering approach commenced with a literature review and structural analysis of the ISO/IEC 27001:2022 and SOC 2 Type II regulatory frameworks. By deconstructing abstract governance mandates into deterministic technical variables, the development team engineered a structured schema capable of mapping operational telemetry to compliance objectives. Following the establishment of this baseline, the methodology emphasized the rapid prototyping of API integration layers. The team evaluated the Wazuh Security Configuration Assessment (SCA) module's capabilities and designed specialized client connectors to poll and serialize distributed endpoint states without overloading the central SIEM infrastructure.

1.4.2 Sandbox Testing and Evaluation

Subsequent development sprints focused on the architectural design and implementation of the backend orchestration system. The methodology prioritized performance optimization by benchmarking database aggregation strategies against synthetic telemetry datasets, resulting in the adoption of $O(1)$ set-based query projections to eliminate relational bottlenecks. In the evaluation phases, the platform transitioned into a containerized deployment model utilizing Docker virtual bridge networks. This isolated environment allowed the team to perform security validation, performance profiling, and stress testing against simulated threat vectors and compliance drift scenarios. By testing the platform within a zero-trust execution sandbox, the methodology ensured the ICMS delivered verifiable security resilience prior to enterprise integration.

1.5 Project Scope

1.5.1 Compliance Framework Boundaries

The ICMS platform operates within the boundaries of the ISO/IEC 27001:2022 and SOC 2 Type II compliance frameworks. The scope encompasses the ingestion and mapping of endpoint security telemetry from the Wazuh Manager's Security Configuration Assessment (SCA) module. By utilizing the Wazuh REST API, the system retrieves check results from active agents and correlates them to identified governance controls. This data mapping ensures the compliance state represents the technical environment's configuration. The system initiates authenticated requests to the SCA module to retrieve payloads describing the pass or fail status of configuration checks on monitored agents. Each retrieved check ID is cross-referenced against the ICMS seed matrix, which acts as a mapping key between technical identifiers and governance controls. This correlation translates technical events into regulatory violations.

1.5.2 Technical Implementation Boundaries

The implementation boundary restricts the system to a containerized Docker ecosystem for isolated operation. This environment utilizes a private virtual bridge network to facilitate communication between the PostgreSQL database, Redis message broker, Celery workers, and the Django-React application stack. This architectural scope ensures the data persistence tier remains inaccessible to external networks, maintaining a baseline for storing compliance telemetry and organizational policies. Several operational domains remain outside the project scope to maintain focus on Governance, Risk, and Compliance (GRC) monitoring. The system does not perform active Incident Response (IR) actions, such as isolating compromised endpoints or terminating malicious processes. The platform does not conduct network packet sniffing or automated remediation of configuration drift. The system evaluates and reports on the compliance state but does not mutate endpoint configurations; infrastructure remediation remains the responsibility of dedicated IT operations teams.

1.6 Feasibility Study

1.6.1 Technical and Economic Feasibility

The implementation of the ICMS platform is technically and economically feasible. Technically, the platform leverages the open-source Wazuh SIEM ecosystem alongside the Django and React frameworks, ensuring architectural dependencies can handle telemetry streams. Containerizing the application stack using Docker ensures deployment across heterogeneous hardware environments. By decoupling the application logic from the host operating system, the system guarantees deterministic execution and allows organizations to deploy the solution on existing infrastructure without specialized hardware procurement. Economically, the platform reduces costs compared to proprietary Software-as-a-Service (SaaS) compliance solutions. By utilizing an open-source technology stack, the system eliminates recurring licensing fees and vendor lock-in risks associated with commercial GRC vendors. Organizations can achieve continuous controls monitoring utilizing existing open-source SIEM infrastructure, shifting the financial model from high operational expenditure (OpEx) to a maintenance profile.

1.6.2 Operational Feasibility

Operationally, the interface and automated telemetry mapping reduce the manual labor hours required for evidence collection, increasing the operational efficiency of Security Operations Centers (SOC).

1.7 Areas of Application

1.7.1 Financial Technology (FinTech)

The ICMS platform provides continuous monitoring capabilities for industries constrained by data privacy and governance mandates. In the Financial Technology (FinTech) sector, where transaction processing and infrastructure scaling outpace point-in-time auditing, the platform delivers the assurance required to satisfy regulatory bodies. FinTech startups processing payment data can utilize the platform to maintain adherence to SOC 2 Type II and internal security policies, mitigating the risk of financial penalties or loss of operational licenses due to undetected configuration drift in cloud environments.

1.7.2 Healthcare Sector

The platform also applies to the Healthcare sector for organizations managing Protected Health Information (PHI). Healthcare organizations validate their infrastructure against frameworks like HIPAA and ISO 27001 to prevent data breaches. The platform automates the translation of technical vulnerabilities into compliance violations, allowing hospital networks to maintain a deterministic posture across distributed endpoints. By providing data sovereignty within an isolated Docker environment, the platform ensures operational telemetry remains within the organization's private network, supporting entities operating under confidentiality requirements.

1.8 Report Organization

This report consists of seven chapters detailing the design and implementation of the ICMS platform. Chapter 2 evaluates literature regarding Governance, Risk, and Compliance (GRC) frameworks and Security Information and Event Management (SIEM) integration. Chapter 3 defines the functional and non-functional requirements and establishes the system architecture. Chapter 4 provides the system design, incorporating Unified Modeling Language (UML) diagrams to represent logical and physical data flows. Chapter 5 focuses on implementation, detailing backend algorithms and containerized orchestration. Chapter 6 outlines testing protocols and evaluates performance against accuracy and latency metrics. Chapter 7 summarizes the project's contributions and proposes directions for future work.

Chapter 2

Literature Review

2.1 Comprehensive Literature Review

2.1.1 Evolution of Governance, Risk, and Compliance (GRC) Platforms

Historical security auditing relied on point-in-time assessments, leaving temporal blind spots between evaluation periods. Organizations managed Governance, Risk, and Compliance (GRC) through manual evidence collection, rendering the compliance state obsolete following an assessment. Regulatory frameworks require a transition toward continuous controls monitoring architectures. Standards including NIST Special Publication 800-53 Revision 5 and ISO/IEC 27001:2022 mandate ongoing monitoring to validate control implementation [8, 2]. The Federal Information Security Modernization Act (FISMA) and the Federal Risk and Authorization Management Program (FedRAMP) require agencies to automate the tracking of systemic anomalies [13]. The transition to SIEM-driven compliance introduces operational advantages. Centralized log management supports audit accountability by preserving data confidentiality, integrity, and availability [4]. Automated analysis establishes operational baselines, facilitating forensic investigations during security incidents. Security administrators configure SIEM agents to track software inventories, hardware configurations, and system health telemetry, feeding this data into the GRC database. Continuous validation replaces reactive audits with a data-driven security posture. By evaluating log streams against established baselines, organizations ensure evidence generation becomes a byproduct of daily security operations.

2.1.2 Security Information and Event Management (SIEM) Integrations

The integration of Security Information and Event Management (SIEM) systems alters the GRC methodology. SIEM platforms act as telemetry ingestion engines, replacing

manual log retrieval with automated data correlation. NIST SP 800-53 defines control families, notably Audit and Accountability (AU) and System and Information Integrity (SI), which require automated tracking to satisfy continuous validation requirements [8]. Automating these controls involves mapping endpoint telemetry directly to regulatory requirements. Platforms like Wazuh, an open-source SIEM and Extended Detection and Response (XDR) solution, facilitate this translation by supplying event aggregation logic that maps compliance events against NIST and ISO matrices [14]. Automated compliance mapping bridges the semantic gap between technical events and business risk. When a user modifies a protected configuration file, File Integrity Monitoring (FIM) sensors generate an event log. The SIEM correlates this event against framework matrices, flagging the action as a violation of ISO 27001 Annex A.8.9 (Configuration Management) or NIST SP 800-53 control CM-6 [3]. Analysts monitor deviations via dashboards that aggregate alert volumes, isolating non-compliant agents for remediation. The Wazuh Security Configuration Assessment (SCA) module provides the deterministic validation required to satisfy these governance frameworks by evaluating configuration states against security benchmarks.

2.1.3 Performance Bottlenecks in Telemetry Aggregation

Calculating compliance metrics across distributed fleets exposes data aggregation limitations in traditional GRC platforms. Standard relational database implementations suffer from the $N+1$ query problem, where the application executes sequential retrieval loops for individual security controls. This architectural flaw imposes a linear $O(N)$ algorithmic complexity on data retrieval, causing geometric performance decay as monitored agents increase [11]. The resulting latency prevents organizations from achieving continuous monitoring, because dashboard rendering times exceed thresholds for operational visibility. The overhead associated with Object-Relational Mapping (ORM) frameworks extends beyond query volume. Application-level data processing, where the system retrieves full object graphs before filtering or aggregating in the Python memory space, creates memory bandwidth bottlenecks. This pattern forces the application server to instantiate transient objects, triggering garbage collection cycles that block the execution of compliance logic. In environments where SIEM agents transmit telemetry continuously, the cumulative impact of these $O(N)$ operations leads to a loss of operational visibility.

Optimizing relational data access requires transitioning from sequential $O(N)$ looping to $O(1)$ set-based aggregations. Set-based modeling formalizes relational dynamics, demonstrating that task-informed database representations retain relevant signals more efficiently than application-level iteration [5]. Developers optimize ORM queries by utilizing

eager loading mechanisms that translate application logic into SQL JOIN clauses. Fetching related entities in a single query allows the database engine’s internal query optimizer to utilize B-Tree indices efficiently, reducing total execution time and minimizing context switching between the application runtime and the database server. Set-based database aggregations shift the mathematical workload to the relational engine. Using query projection techniques, the system instructs the database to compute sums, averages, and conditional compliance counts internally. This methodology reduces data transfer across the network wire from megabytes of row data to primitive integer values. By utilizing conditional aggregation functions, where the engine evaluates compliance criteria during the initial table scan, the platform achieves a deterministic $O(1)$ response time. Implementing these optimizations ensures dashboard rendering remains responsive as the organizational asset inventory scales. The architectural decision to prioritize database-side computation over application-side iteration reflects the principle of pushing logic closer to the data. This approach ensures the GRC platform can process telemetry logs without exhausting the system’s computational resources.

2.1.4 Modern Access Control Paradigms

Securing the compliance data layer against unauthorized extraction requires Identity and Access Management (IAM) protocols. Role-Based Access Control (RBAC) provides the foundation for authorization within distributed systems. RBAC methodologies control access to system resources based on roles assigned to a user, rather than assigning explicit permissions directly to the user identity [9]. Authentication in distributed microservices relies on stateless session security, primarily implemented via JSON Web Tokens (JWT). The authorization server cryptographically signs the JWT using asymmetric encryption algorithms, preventing payload tampering and allowing local verification without initiating a database lookup. Authorization in the ICMS platform relies on an enforcement layer that integrates Zero Trust principles into the API routing logic. Rather than assuming internal network trust, the system evaluates the identity context of every inbound request. Custom permission classes intercept traffic at the view layer, executing Boolean logic to verify the requester possesses the RBAC attributes required for the target resource. This enforcement remains decoupled from the business logic, ensuring a consistent security boundary across the microservice ecosystem.

The security lifecycle of stateless JSON Web Tokens (JWT) incorporates rotation policies to mitigate risks inherent in long-lived credentials. Traditional session IDs require database lookups to verify validity; JWTs carry cryptographically signed claims that allow microservices to authenticate users locally. To prevent session hijacking via inter-

cepted tokens, the platform implements a refresh token rotation scheme. When the client requests a new access credential, the server invalidates the previous refresh token and issues a new pair, ensuring compromised credentials have limited utility. The architecture incorporates Time-Based One-Time Passwords (TOTP) as a secondary authentication factor. TOTP algorithms rely on a high-entropy, shared cryptographic secret established between the server and the client’s authenticator application. Both entities use the current Unix timestamp, divided by a predetermined time interval, as the moving factor in a Keyed-Hash Message Authentication Code (HMAC-SHA1) function [7]. Because the resulting 6-digit numeric code mutates deterministically every 30 seconds, intercepted credentials become mathematically useless. Secondary authentication factors establish an additional defensive layer that remains resilient against replay attacks. Client-side interceptors monitor for specific HTTP response states; when the server returns a forbidden status due to a missing or invalid TOTP challenge, the application redirects the user to a step-up authentication view, preventing unauthorized access. This session lifecycle ensures the compliance data remains inaccessible to compromised endpoints.

2.1.5 Containerization and Defense-in-Depth Topology

Enterprise architecture increasingly transitions from monolithic software design toward microservices and containerized deployments. Monolithic systems couple application logic, user interfaces, and database drivers into a single deployment artifact, where a failure in one component frequently cascades across the runtime environment. Containerization technologies, specifically Docker, provide the standardized runtime environment for these distributed microservices, ensuring deterministic scalability and bounded contexts through Linux kernel namespaces and control groups. Deploying the ICMS components via isolated Docker virtual bridge networks establishes a defense-in-depth topology. The architecture physically separates the public-facing API gateway layers from the private persistent storage layers, such as PostgreSQL and Redis. By binding these data tiers to internal, non-routable Docker bridge networks, the system remains inaccessible to external network scans. This isolation ensures actors cannot establish direct TCP connections with the database, forcing data interactions through the authenticated business-logic layer. Adherence to external hardening standards, such as NIST SP 800-190 and the CIS Docker Benchmark, further neutralizes execution environment vulnerabilities [12, 1].

2.1.6 Cognitive Load and HCI in Security Dashboards

Security Operations Centers process large volumes of diagnostic data daily, frequently triggering alert fatigue. Legacy GRC platforms present analysts with unfiltered data in dense HTML tables, leading to sensory overload and an increase in diagnostic errors. High cognitive load correlates with delayed incident response times and operator burnout [15]. Mitigating cognitive strain requires the application of User Interface (UI) design principles based on human-computer interaction studies. Modern browser rendering engines deprecate structural tables for layout purposes in favor of CSS Flexbox and Grid modules, which allow child components to wrap and scale fluidly based on viewport dimensions [6]. The ICMS platform utilizes modular card components and semantic color scaling, such as bold red for critical exploits and minimal grayscale for benign logs, to guide the analyst’s focus toward critical threats. Leveraging elevation through depth illusions distinguishes interactive components from static background data, structuring telemetry into a decision-support matrix.

2.2 Analysis of Existing Systems

2.2.1 Manual Spreadsheet Auditing

Evaluating the baseline of traditional governance reveals operational limitations inherent in manual compliance methodologies. Historically, organizations relied on labor-intensive processes encompassing decentralized spreadsheets, task-tracking tickets, and point-in-time audits conducted by external consultants. This fragmented approach fails to scale within IT environments, because the compliance state captured during an audit becomes obsolete upon the conclusion of the assessment. The reliance on manual sampling and evidence collection introduces temporal blind spots, where vulnerabilities remain undetected between audit cycles. The structural limits of spreadsheet applications lead to performance degradation and data corruption when tracking large volumes of configuration parameters. Manual mapping processes introduce a high probability of human error, as analysts attempt to interpret technical data against regulatory constraints.

2.2.2 Vanta (Commercial SaaS)

Vanta operates in the commercial Software-as-a-Service (SaaS) continuous monitoring sector. The platform accelerates the evidence collection process by deploying propri-

etary agents and utilizing API connectors to integrate with public cloud providers and enterprise identity systems. This commercial solution introduces risks regarding data sovereignty and vendor lock-in. Operating as a multi-tenant cloud environment, Vanta requires client organizations to transmit security telemetry to its external, third-party infrastructure. This data exfiltration increases the attack surface, introducing potential exposure via shared infrastructure vulnerabilities. Vanta operates as a proprietary system; its closed-source algorithms prevent client organizations from independently verifying or customizing how raw telemetry events map to compliance controls. This lack of transparency forces organizations to rely on the provider’s internal security assertions, contradicting the principles of algorithmic transparency.

2.2.3 Drata (Commercial SaaS)

Drata provides automated compliance tracking via a centralized, multi-tenant SaaS architecture. While the platform offers integrations and pre-mapped framework tracking, it involves risks regarding external data storage. Organizations utilizing Drata export their infrastructure metadata to the provider’s cloud, violating data sovereignty requirements mandated by entities in the defense and financial sectors. The closed-source nature of Drata restricts security engineers from inspecting the underlying evaluation logic. This limitation prevents custom programmatic modifications to the compliance engine, meaning that organizations with unique internal architectures or non-standard security tools cannot extend the system without relying on proprietary vendor interventions. The inability to inspect the evaluation engine ensures the platform remains a closed-box compliance oracle rather than a verifiable security tool.

2.2.4 AuditBoard (Enterprise GRC)

AuditBoard targets the enterprise GRC market, focusing on risk management, ESG reporting, and internal audit synchronization. Despite its feature set, AuditBoard introduces integration rigidity. The platform demands high licensing costs and requires professional service engagements for initial deployment, resulting in deployment cycles that frequently span multiple financial quarters. This enterprise architecture compares poorly against containerized solutions that deploy rapidly. AuditBoard relies on proprietary connectors to interface with security tools, establishing a closed-box mapping approach. Security engineers lack the capacity to manipulate the programmatic logic linking SIEM alerts to organizational risk matrices. This opacity limits the platform’s utility in environments that demand deterministic, verifiable compliance states.

2.3 Comparative Analysis

The ICMS platform establishes an alternative for organizations requiring data sovereignty, technical transparency, and verifiable execution logic. By integrating with the open-source Wazuh SIEM, the platform achieves automation without necessitating external data transmission. The architecture provides programmatic control via the open-source Django framework, allowing organizations to independently audit, modify, and extend the underlying compliance evaluation algorithms. The single-tenant Docker topology ensures that security telemetry remains within an isolated, customer-controlled bridge network, neutralizing the attack surface associated with multi-tenant cloud environments.

As detailed in Table 2.1, the ICMS addresses the performance decay observed in proprietary systems through database engineering. By implementing $O(1)$ set-based aggregations via query annotation and SQL filtering, the platform processes telemetry logs efficiently. This deterministic execution ensures the platform maintains lower latency compared to legacy systems burdened by monolithic database scaling constraints.

Table 2.1: Comparative Feature Matrix: Analyzing GRC Platforms

Feature	Traditional Methods (Manual)	Commercial SaaS (Vanta / Drata / AuditBoard)	ICMS Platform
Deployment Model	Decentralized documents	Multi-tenant public cloud (SaaS risk)	Single-tenant, isolated Docker bridge
Data Sovereignty	High (Data remains local)	Low (Data transmitted to third parties)	High (Data confined to internal network)
Performance Complexity	Prone to spreadsheet limits and crashes	Proprietary ORM scaling constraints, API latency	$O(1)$ set-based aggregation via <code>.annotate()</code>
Customizability	Manual modification	Rigid black-box mapping algorithms	Full programmatic control via Django ORM
Cost	High operational labor expenditure	Expensive recurring enterprise licensing	Zero licensing cost (Open-source stack)

2.4 Summary of Competitive Advantages

The ICMS platform provides operational and technical advantages that address the flaws of legacy auditing and commercial SaaS offerings:

- **Data Sovereignty via Docker Bridge Isolation:** The platform confines all PostgreSQL database interactions to a private virtual network, ensuring compliance telemetry remains isolated from the public internet and external third-party vendors.
- **Transparent, Verifiable Mapping:** By utilizing an open-source Django backend, organizations can programmatically inspect and verify the deterministic logic linking raw Wazuh SIEM alerts to governance controls, eliminating the proprietary closed-box approach.
- **Deterministic $O(1)$ Database Performance:** Set-based aggregations replace standard ORM looping structures, allowing the system to render fleet-wide compliance dashboards with low latency regardless of the agent inventory size.
- **Zero-Cost Licensing Architecture:** The operational stack relies on open-source technologies (Wazuh, PostgreSQL, Python), removing the financial burden of recurring commercial SaaS fees and professional deployment services.
- **Automated Telemetry Deduplication:** The implementation of a Worst-Case Priority algorithm ensures that conflicting telemetry reports from disparate sensors are resolved deterministically, prioritizing organizational safety and preventing false-positive compliance states.

Chapter 3

Requirement Specification

3.1 Existing System

Legacy compliance management systems rely on manual spreadsheet auditing and fragmented Software-as-a-Service (SaaS) platforms. Governance teams attempt to map regulatory standards, such as ISO/IEC 27001:2022, to technical controls using decentralized, static documents. This approach introduces human error, lacks version control, and fails to maintain referential integrity across overlapping frameworks. The manual tracking of configuration states results in heavy operational labor expenditures and administrative burden.

Existing point-in-time auditing solutions perform delayed security assessments, meaning data collected from internal network systems becomes stale upon generation. Legacy platforms employ generic software flows lacking specialized relational constraints, leading to latency when processing endpoint telemetry and an inability to maintain determinism in vulnerability resolution. The reliance on proprietary SaaS architectures introduces data sovereignty risks, because internal compliance metrics and infrastructure configurations are transmitted to multi-tenant public cloud environments.

3.2 Proposed System

To overcome the limitations of legacy spreadsheet mapping and asynchronous SIEM aggregation, the ICMS platform introduces a centralized, deterministic orchestration engine. The system leverages a multi-tier architectural topology designed to maintain separation of concerns while providing compliance tracking within a single-tenant, isolated environment.

3.2.1 Technology Stack Selection

Selecting the Python-based Django framework for the backend application layer provides an environment characterized by an integrated Object-Relational Mapper (ORM) and native security middleware. The Django ORM facilitates the implementation of performance optimizations, specifically $O(1)$ set-based database aggregations. By utilizing query projections and conditional filtering at the ORM level, the system ensures high-volume telemetry remains computationally manageable. The Django REST Framework (DRF) extends these capabilities by providing modular serialization pipelines and support for authentication schemas, including stateless JSON Web Token (JWT) rotation.

The selection of React 18 and the Vite build tool establishes a Single Page Application (SPA) architecture designed for data visualization. React's Virtual DOM provides an advantage in the context of security monitoring, allowing the interface to render compliance state changes without causing browser thread execution delays. This reactive rendering ensures analysts maintain an operational experience when viewing dense matrices of SIEM alerts. Vite enhances the development and production lifecycle by utilizing native ES modules and Hot Module Replacement (HMR), reducing build latency compared to legacy Webpack-based bundling strategies.

PostgreSQL serves as the primary data persistence tier, providing the ACID compliance required for maintaining the integrity of organizational compliance records. The relational engine handles the concurrent read/write loads generated by SIEM telemetry ingestion and automated framework mapping. By utilizing B-Tree indexing and relational constraints, PostgreSQL guarantees cross-framework compliance metrics remain accurate. This data integrity satisfies the evidence requirements of ISO 27001 and SOC 2 audits, where the historical state of security controls must remain verifiable and immutable.

To ensure the platform remains responsive during data-heavy operations, the architecture incorporates Celery and Redis as the asynchronous orchestration layer. Decoupling I/O-bound tasks, such as the periodic synchronization of telemetry from the Wazuh SIEM REST API, from the main Django application thread prevents HTTP request timeouts and maintains dashboard availability. Redis acts as a message broker, facilitating the transport of task payloads between the web server and the background worker fleet. This distributed task execution model allows the platform to scale its ingestion capacity without impeding the interaction capabilities of the primary API service.

3.2.2 High-Level Architecture

The Presentation Tier consists of a React-based SPA that manages the user session and provides an interface for compliance monitoring. This tier communicates with the Application Logic Tier via stateless JSON payloads over an encrypted HTTPS channel. By offloading the state management and rendering logic to the client-side browser environment, the system ensures central server resources remain available for data processing and framework mapping tasks.

The Application Logic Tier, powered by the Django REST Framework, serves as the gateway for compliance orchestration. This tier manages the Identity and Access Management (IAM) lifecycle, including the generation and rotation of JSON Web Tokens (JWT). Interactions with the Data Persistence Tier occur via TCP connections managed by the Django ORM, which enforces relational integrity and executes query aggregations. Asynchronous task execution relies on a distributed architecture incorporating Celery and a Redis-based message broker. The system utilizes a Pub/Sub messaging model to enqueue compliance synchronization tasks, allowing Celery workers to process telemetry data from the external Wazuh SIEM Manager out-of-band. Figure 3.1 maps the protocol flow and multi-tier layout.

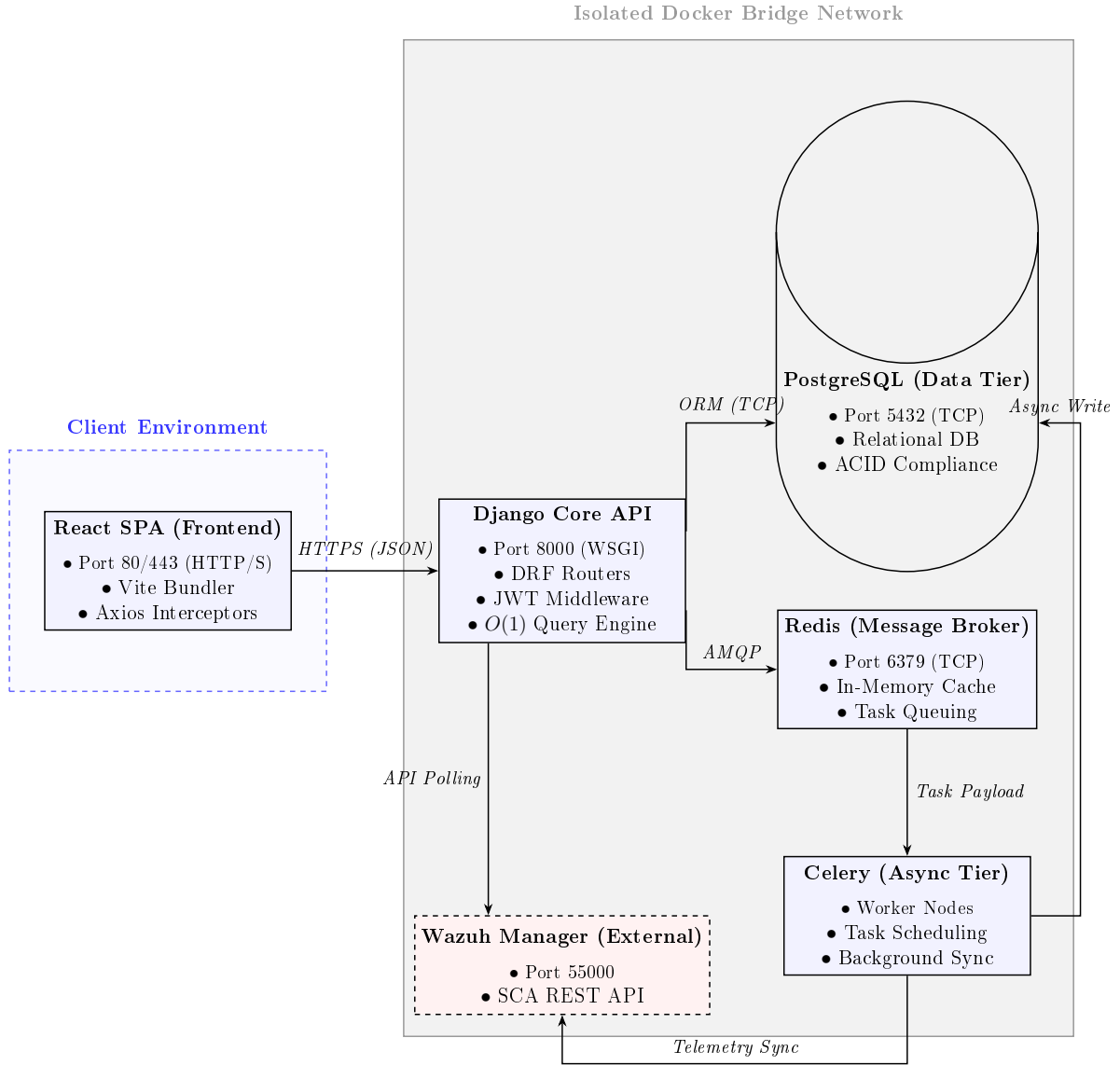


Figure 3.1: Refined N-Tier System Architecture and Protocol Flow

3.2.3 Dockerized Infrastructure and Network Isolation

The implementation of a Dockerized virtual bridge network establishes a perimeter that isolates the data persistence tier from external network environments. By restricting the PostgreSQL and Redis services to an internal, non-routable network, the architecture ensures these components remain inaccessible to host-level scans. Managing data persistence across container lifecycles requires Docker volumes to prevent the loss of compliance telemetry. Orchestrating the system startup sequence through deterministic health-check dependencies ensures the platform remains resilient against race conditions during deployment. The configuration defines execution priorities, where the Django API container monitors the execution of system-level commands before initiating database

migrations or starting the WSGI server. Figure 3.2 depicts this multi-tier isolation and health monitoring network.

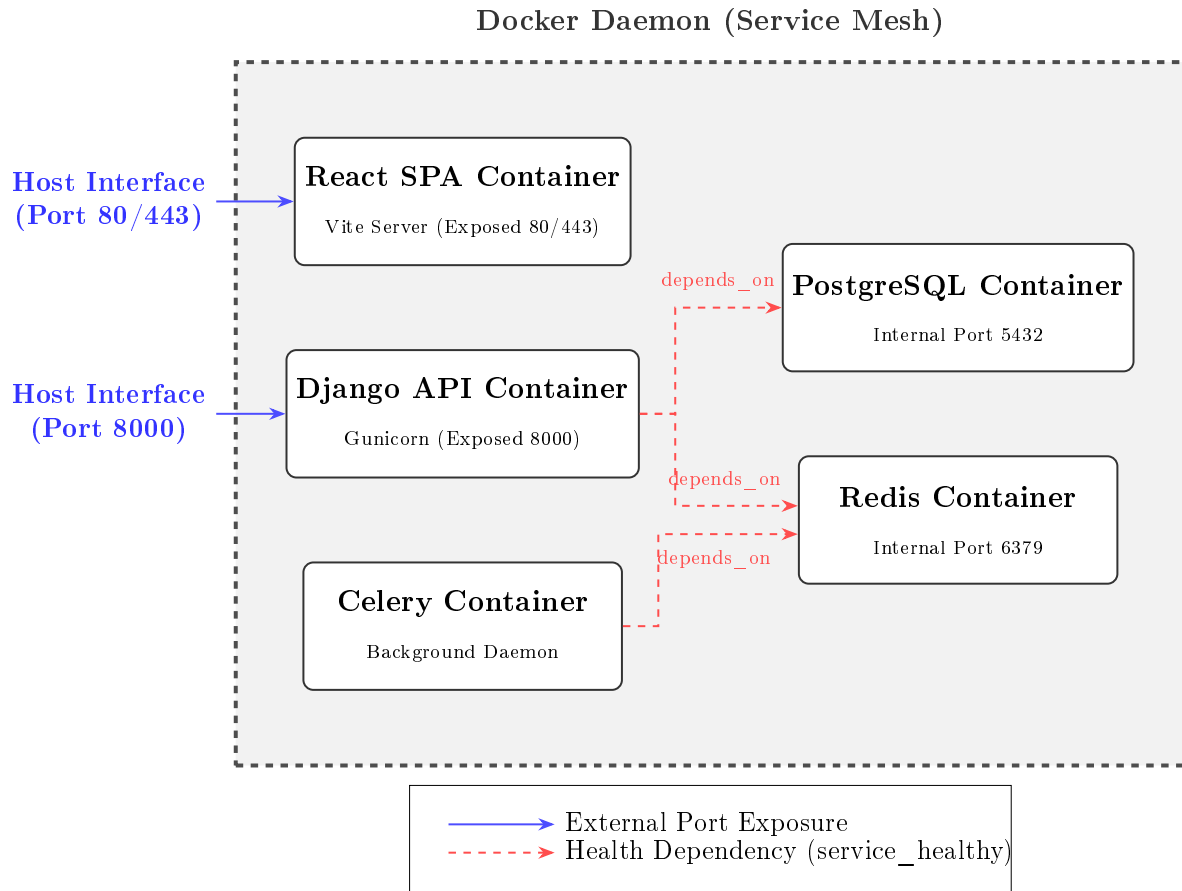


Figure 3.2: Docker Service Mesh: Port Exposure and Health-Check Dependencies

3.3 Requirements Specification

Requirement engineering for the ICMS platform involves mapping regulatory mandates to technical specifications. Aligning the platform with ISO/IEC 27001:2022 and SOC 2 Type II standards requires the translation of governance objectives into deterministic functional capabilities. This process ensures architectural components satisfy compliance or operational constraints. The requirement analysis phase establishes the foundation for a system capable of maintaining visibility across distributed infrastructure by prioritizing automated telemetry ingestion and data aggregation.

3.4 Functional Requirements

The functional requirements define the capabilities the ICMS architecture must possess to execute its core operations:

- The system must enforce a 3-tier Role-Based Access Control (RBAC) model integrated with Time-Based One-Time Password (TOTP) secondary authentication.
- The platform must automate the ingestion of endpoint security telemetry from the Wazuh SCA module via authenticated REST API synchronization.
- The system must resolve conflicting telemetry states using a deterministic Worst-Case Priority state-mutation algorithm to ensure data integrity.
- The platform must generate server-side compliance reports in PDF format utilizing CSS2.1 compliant layouts for audit documentation.
- The system must provide an administrative interface for the dynamic management of frameworks, controls, and Wazuh SIEM mapping identifiers.
- The application must implement a stateless session management lifecycle utilizing JSON Web Token (JWT) rotation for all authenticated requests.

3.5 Non-Functional Requirements

The non-functional requirements dictate the operational constraints and performance thresholds required to maintain system reliability:

- The system must isolate the PostgreSQL persistence tier and Redis message broker within a private Dockerized virtual bridge network.
- The backend must utilize $O(1)$ set-based query aggregations to ensure dashboard rendering latency remains below 200 milliseconds.
- The architecture must employ asynchronous background orchestration via Celery and Redis to maintain non-blocking UI responsiveness during data ingestion.
- The frontend must utilize a responsive Flexbox and Cyber-Card based architecture to mitigate analyst alert fatigue and reduce cognitive load.

- The platform must ensure high availability through the implementation of Celery task retry logic and persistent message queuing for all telemetry sync tasks.
- The system must maintain a granular audit logging engine that records all administrative actions and automated system events for forensic traceability.

3.6 User Requirements

The operational efficacy of the ICMS platform relies on delivering a user experience tailored to its primary actors: the System Administrator and the Security Auditor. The System Administrator requires a control plane capable of managing infrastructure settings without exposing database commands. They must possess the capability to configure the synchronization lifecycle, manage RBAC privileges across user accounts, and override automated telemetry when necessary. The interface must present these administrative controls with a low cognitive load, utilizing visual feedback through the component library to confirm successful CRUD operations within the Django ORM.

The Security Auditor interacts with the platform as an analytical tool to assess regulatory adherence. Their primary requirement is parsing compliance data without the visual clutter associated with legacy GRC software. The auditor requires an RBAC boundary that prevents modification of infrastructure mapping while offering access to historical scan logs and PDF export functionalities. The React SPA architecture must guarantee the dashboard remains responsive during concurrent data ingestion from the Wazuh SIEM, employing memoization to ensure DOM updates do not introduce browser thread execution delays.

3.7 System Models and Components

This section outlines the systemic data exchange boundaries and flow orchestrations utilized by the platform to synchronize external SIEM telemetry with the internal compliance tracking engine.

3.7.1 Context Diagram

The Context Diagram defines the interaction between the ICMS core logic and external entities, illustrating the boundaries of the application and the data streams required for

operation. Figure 3.3 maps the interaction flow from human operators to infrastructure components.

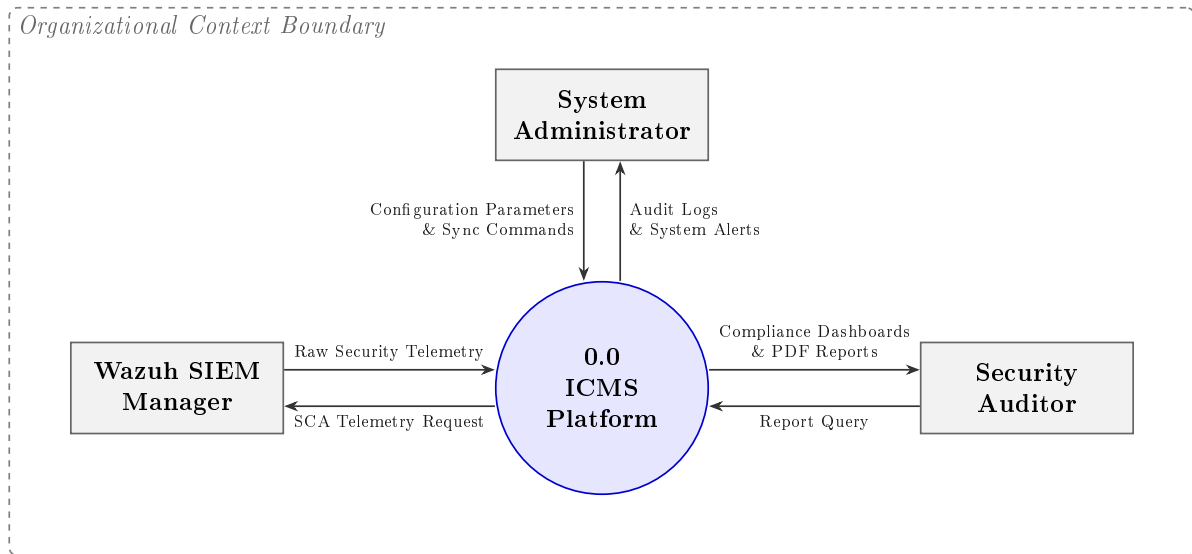


Figure 3.3: Context Diagram: High-Level System Interactions

3.7.2 Data Flow Diagram

The Level-1 Data Flow Diagram (DFD) offers a visualization of how data payloads traverse the platform’s internal architecture, mapping the transformation of raw SIEM logs into structured compliance metrics. Figure 3.4 outlines the deterministic pipeline executed by the ICMS.

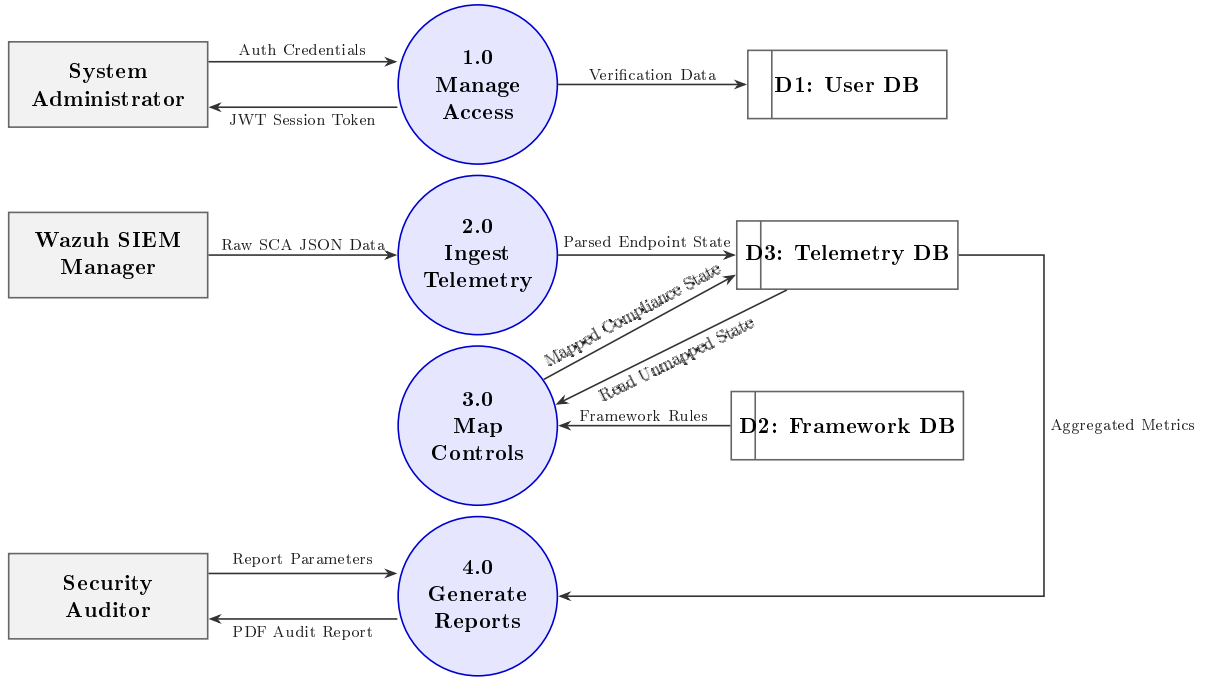


Figure 3.4: Level-1 Data Flow Diagram detailing internal compliance processes.

3.8 Use Cases

The use cases define the interaction sequences available to authenticated actors, providing deterministic state definitions for platform operations.

3.8.1 Use Case Diagram

Figure 3.5 maps the operational workflows orchestrated by the System Administrator and the Security Auditor, establishing boundaries of execution based on the platform's internal RBAC implementation.

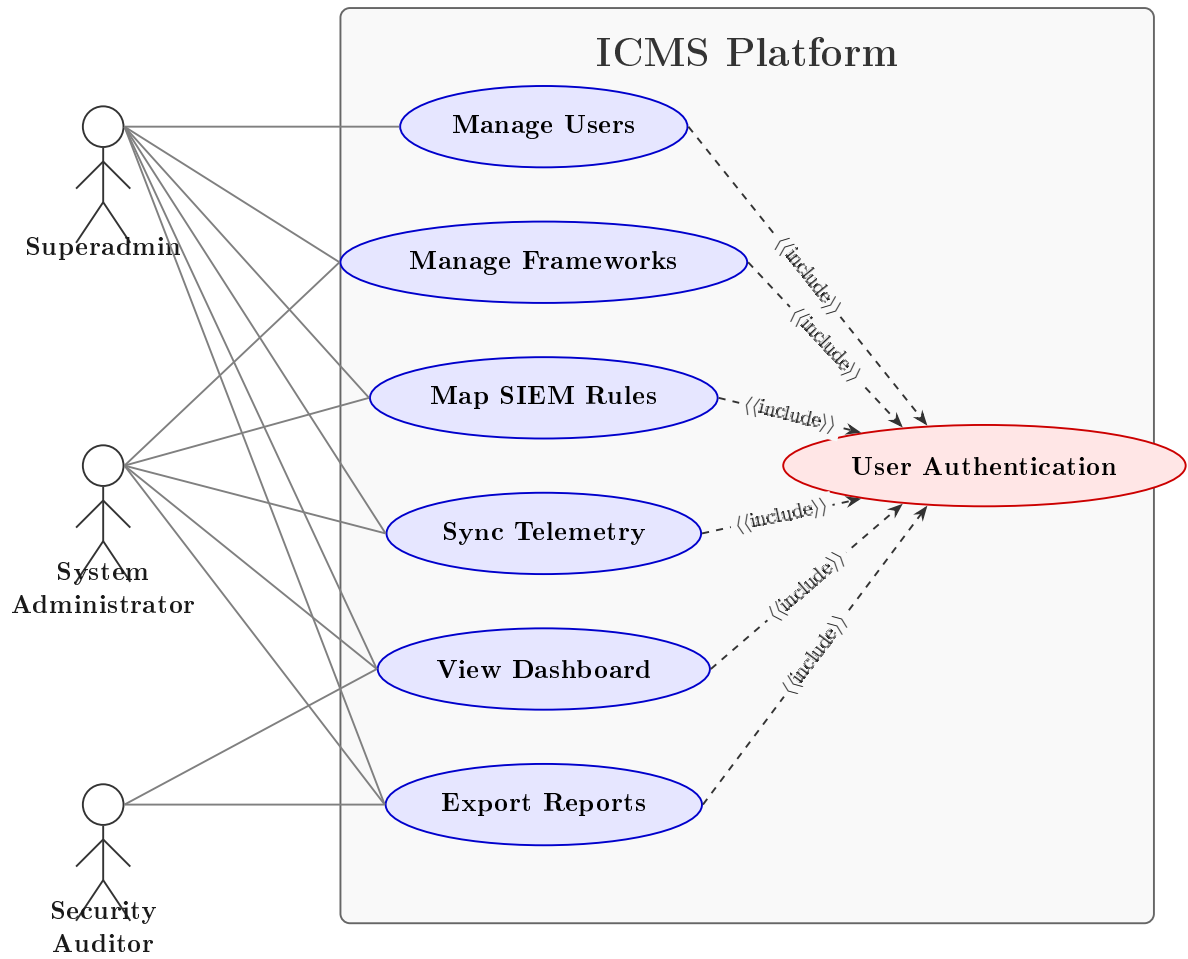


Figure 3.5: System Use Case Diagram

3.8.2 Descriptive Use Cases

The following tabular specifications define the algorithmic flows, authentication prerequisites, and resulting persistence states for each operation.

Table 3.1: Use Case - Authentication

Attribute	Description
Use Case ID	UC-01
Use Case Name	User Authentication
Primary Actor	System Administrator, Security Auditor
Preconditions	The user account is registered and active in the system.
Main Flow	1. The user enters their credentials into the login interface. 2. The system verifies the submitted credentials. 3. The system grants access and displays the default dashboard view.
Alternative Flows	Invalid credentials → The system denies access and displays an authentication error message. Session timeout → The system automatically refreshes the session without interrupting the user.
Frequency of Use	High (Initiated upon every new user session or token expiry).
Priority	Critical (Gatekeeper for all subsequent RBAC routing).
Exception	Account locked due to repeated invalid attempts → Deny access, log event to security audit trail, and prompt system administrator intervention.
Postconditions	The user is successfully authenticated and an active session is established.

Table 3.2: Use Case - Manage Frameworks

Attribute	Description
Use Case ID	UC-02
Use Case Name	Manage Wazuh Telemetry Synchronization
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. The external SIEM system is reachable.
Main Flow	1. The Administrator triggers a manual synchronization of security telemetry. 2. The system requests audit data from the external SIEM. 3. The system processes and maps the received data against the linked controls. 4. The system updates the compliance status for each control. 5. The system stores the audit results and notifies the Administrator.
Alternative Flows	SIEM system is unreachable → The system retries the connection and alerts the Administrator if it ultimately fails.
Frequency of Use	High (Continuous via Celery beat background tasks; occasional manual ad-hoc triggers by Superadmin).
Priority	Critical (Forms the core data ingestion pipeline for the compliance engine).
Exception	Wazuh REST API connection timeout or TLS handshake failure → Abort sync, log connection error, and queue retry via Celery exponential backoff.
Postconditions	The system reflects the latest compliance state based on the synchronized SIEM data.

Table 3.3: Use Case - Map Controls

Attribute	Description
Use Case ID	UC-03
Use Case Name	Map Framework Controls to Wazuh Rules
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. Frameworks and controls exist in the system.
Main Flow	1. The Administrator selects a SIEM rule and links it to a specific compliance control. 2. The system validates the relationship between the rule and the control. 3. The system saves the mapping configuration. 4. The system confirms the successful linkage to the Administrator.
Alternative Flows	Invalid control selected → The system rejects the mapping and prompts the Administrator to select a valid control.
Frequency of Use	Low/Medium (Executed during initial platform setup or when regulatory frameworks release new versions).
Priority	High (Defines the relational logic for the entire compliance evaluation).
Exception	Mapping collision or invalid/deprecated Wazuh Rule ID → Prevent database commit, display strict validation error to the Superadmin.
Postconditions	A linkage is established between the SIEM rule and the compliance control for future audits.

Table 3.4: Use Case - Sync Telemetry

Attribute	Description
Use Case ID	UC-04
Use Case Name	Monitor Endpoint Compliance Status
Primary Actor	System Administrator (Trigger) / Celery (System Actor)
Preconditions	The user is authenticated. <<include>> User Authentication.
Main Flow	1. The user navigates to the dashboard view. 2. The system aggregates the latest compliance data. 3. The system visualizes the compliance score and detailed control statuses.
Alternative Flows	No data available → The system displays an empty state with a prompt to synchronize telemetry.
Frequency of Use	High (Serves as the primary operational dashboard for continuous monitoring).
Priority	High (Core visibility and monitoring function).
Exception	Endpoint agent disconnected or offline → Mark endpoint status as 'Stale/Offline', visually highlight the disconnection in the UI, and suspend compliance calculations for that node.
Postconditions	The user is presented with an up-to-date visual compliance posture.

Table 3.5: Use Case - View Dashboard

Attribute	Description
Use Case ID	UC-05
Use Case Name	Resolve Compliance Anomalies
Primary Actor	Security Auditor, System Administrator
Preconditions	The Auditor or Admin is authenticated. Compliance failures have been detected in the system.
Main Flow	1. The user identifies a non-compliant control on the dashboard. 2. The user initiates a resolution workflow and applies corrective actions to the endpoint. 3. The system verifies the change and logs the remediation status.
Alternative Flows	Resolution fails → The system maintains the 'Failed' state and prompts for manual investigation.
Frequency of Use	Medium (Event-driven, based on incoming alerts and failed states).
Priority	Medium/High (Critical for operational remediation, but dependent on external IT actions).
Exception	Insufficient RBAC permissions to apply resolution (e.g., Auditor attempting Admin action) → Deny action, return HTTP 403 Forbidden, and log unauthorized attempt.
Postconditions	The compliance anomaly is resolved and recorded in the historical audit ledger.

Table 3.6: Use Case - Export Audit Reports

Attribute	Description
Use Case ID	UC-06
Use Case Name	Export Audit Reports
Primary Actor	Security Auditor
Preconditions	The Auditor is authenticated. $\langle\langle\text{include}\rangle\rangle$ User Authentication. Compliance data exists.
Main Flow	1. The Auditor requests a formal compliance report for a specific framework. 2. The system compiles the historical audit data. 3. The system generates a formatted PDF document. 4. The system prompts the Auditor to download the document.
Alternative Flows	Generation failure $\rightarrow$ The system displays a generation error message and logs the failure.
Frequency of Use	Low (Triggered periodically based on audit cycles, e.g., monthly or quarterly).
Priority	High (Primary executive deliverable for external governance bodies).
Exception	PostgreSQL data retrieval timeout or PDF template rendering failure $\rightarrow$ Abort generation, display generic error modal, and log backend stack trace.
Postconditions	A timestamped audit report is delivered to the Auditor.

Chapter 4

Design

4.1 System Architecture

I designed the Intelligent Compliance Management System (ICMS) using an N-tier architecture. Separating the frontend, API, background workers, and database means we can scale or debug each piece without taking the whole system down. Figure 4.1 breaks down the four main layers.

4.1.1 Client and Presentation Tier

The frontend is a React 18 Single Page Application (SPA). Moving the rendering logic to the browser keeps the server from getting bogged down. It also means analysts don't have to wait for full page reloads when reviewing compliance alerts.

4.1.2 Application and Middleware Tier

The backend runs on the Django REST Framework. This acts as the main gateway, checking JWTs and enforcing role-based access before running any queries. We route all database traffic through Django to maintain a strict security boundary.

4.1.3 Asynchronous Processing Tier

Pulling thousands of logs from the Wazuh SIEM takes time. If we did this on the main Django thread, the API would hang and time out. Instead, I offloaded these I/O tasks to background Celery workers using Redis as the message broker.

4.1.4 Data Persistence Tier

I chose PostgreSQL 18 for the database because compliance data needs strict ACID guarantees. We use standard relational tables for mapping frameworks to controls, but rely on Postgres’s JSONField to store the unstructured SIEM logs.

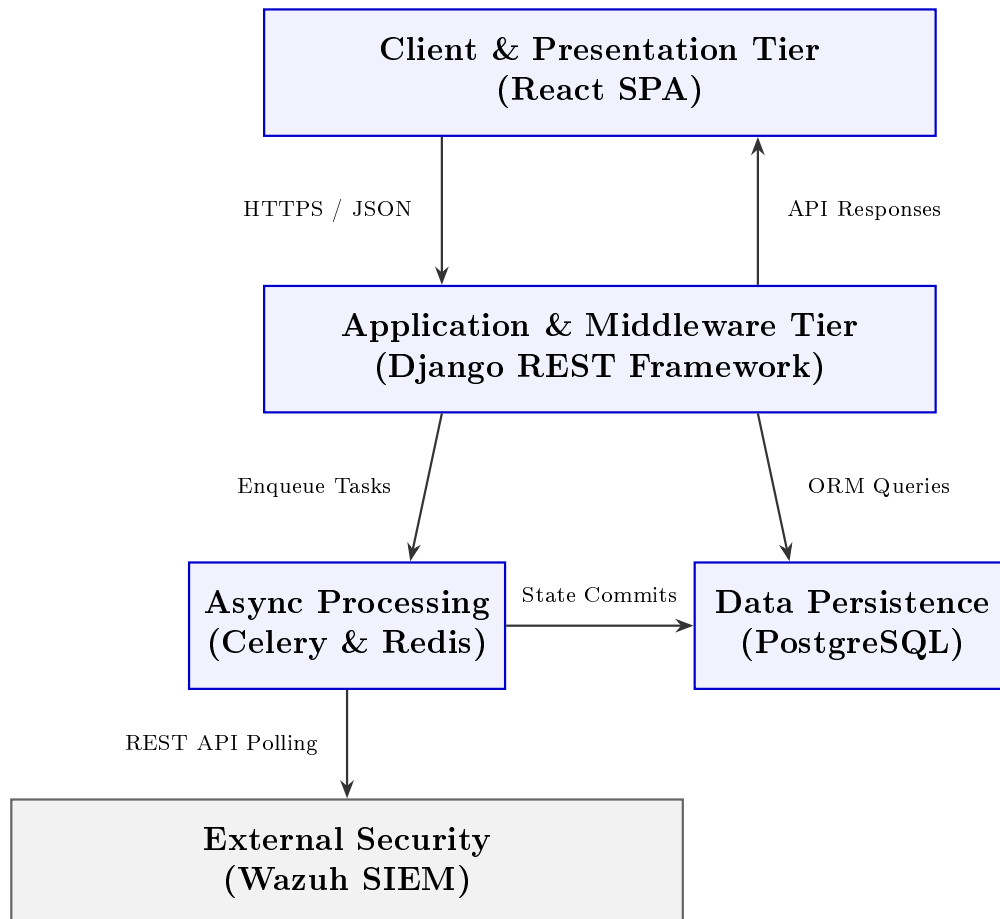


Figure 4.1: Tier-wise System Architecture of the ICMS Platform

4.2 Design Constraints

Building the ICMS wasn’t just about writing code; I had to work around a few hard technical and environmental limits.

4.2.1 Technical Constraints

For security, the database and Redis broker run inside a private Docker bridge network. I didn’t expose their ports to the host machine. If you want data, you have to go through

the Django API container.

Integrating with the Wazuh API introduced rate limits and network latency. The Celery workers have to batch and throttle their polling requests. Because of this, the dashboard shows the compliance state at the last successful sync, rather than a true real-time feed.

On the frontend, rendering a massive grid of compliance checks can crash the browser. Trying to load thousands of DOM nodes at once blocks the main thread. I fixed this by using React virtualization so the UI only renders what's actually visible on the screen.

4.2.2 Infrastructure and Operational Constraints

Early on, my partner and I lost access to the university's lab network. We had to pivot to building and testing the entire enterprise architecture from home on our personal laptops.

Running a full Wazuh SIEM manager locally burned through our RAM and crashed our machines. To get around this, we built a distributed home-lab. I dedicated one laptop entirely to running Wazuh, while my partner used his machine for development and as the monitored endpoint. We relied heavily on Git to keep our codebases synced while the heavy infrastructure stayed on the dedicated server.

4.3 Design Methodology

We built the platform iteratively using Agile prototyping. Instead of trying to build everything at once, we broke the system into distinct modules so we could test each domain independently before merging.

The project was divided into four main sprints.

- **Sprint 1 (Prototyping):** We first needed to prove the core concept worked. I set up a basic Django backend and tested the Wazuh API to make sure we could actually pull Security Configuration Assessment (SCA) data.
- **Sprint 2 (Core Architecture):** Next, I designed the PostgreSQL database schema to link frameworks to rules. We also locked down the API by implementing JWT authentication and role-based routing.
- **Sprint 3 (Asynchronous Telemetry):** Because the SIEM data was too heavy for synchronous requests, I added Celery and Redis. This gave us a background

worker fleet to handle ingestion and deduplication.

- **Sprint 4 (Frontend & Visualization):** Finally, we built out the React frontend. This involved rendering the data grids, wiring up the state management, and setting up the backend PDF generator for audit reports.

4.4 High Level Design

To explain how the ICMS actually works, I mapped the architecture across several different views. These diagrams cover the software components, the physical deployment, and the sequence of operations.

4.4.1 Component Diagram

Figure 4.2 shows how the modules talk to each other. The React frontend sends HTTPS requests to the Django API Gateway. The IAM module checks the JWT, and then the Compliance Engine translates the request into a database query. Off to the side, the Wazuh Polling Service runs independently, pulling alerts and dumping them into PostgreSQL.

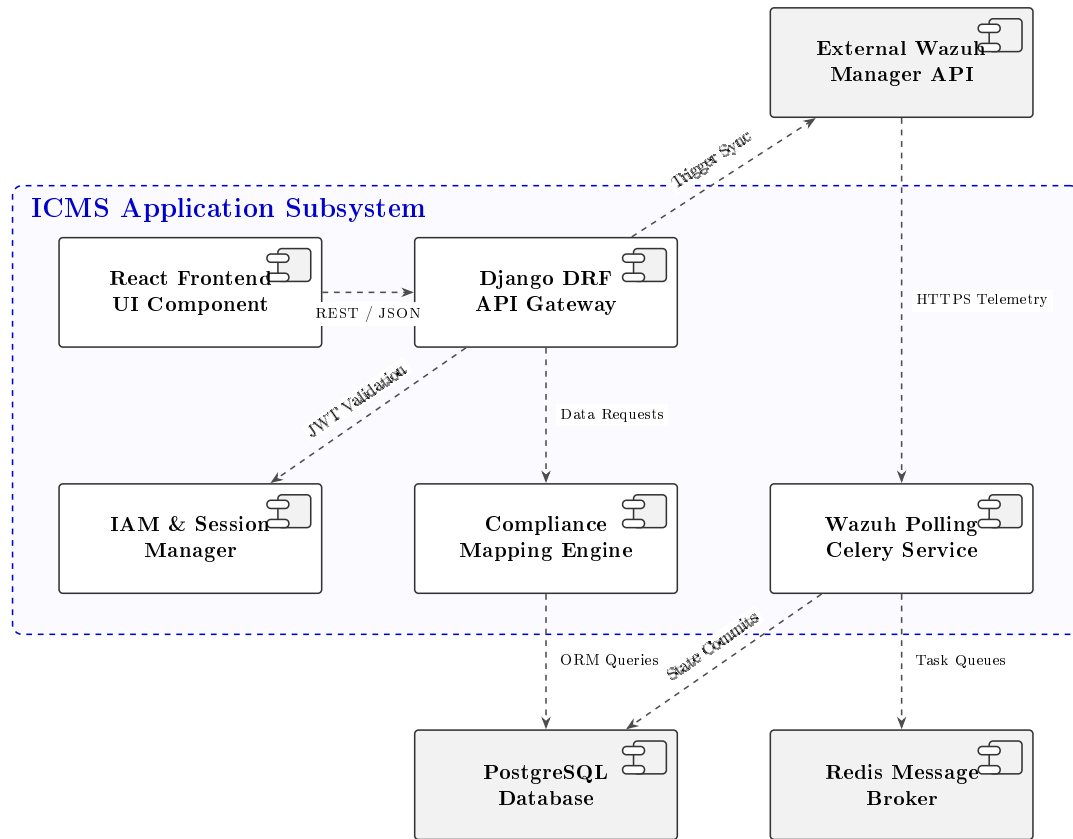


Figure 4.2: Conceptual View: UML Component Diagram

4.4.2 Deployment Diagram

Figure 4.3 lays out the Docker container environment. I put the Django backend, Celery worker, Redis, and PostgreSQL all on the same host but isolated them inside a private bridge network. This means the database is inaccessible from the outside internet.

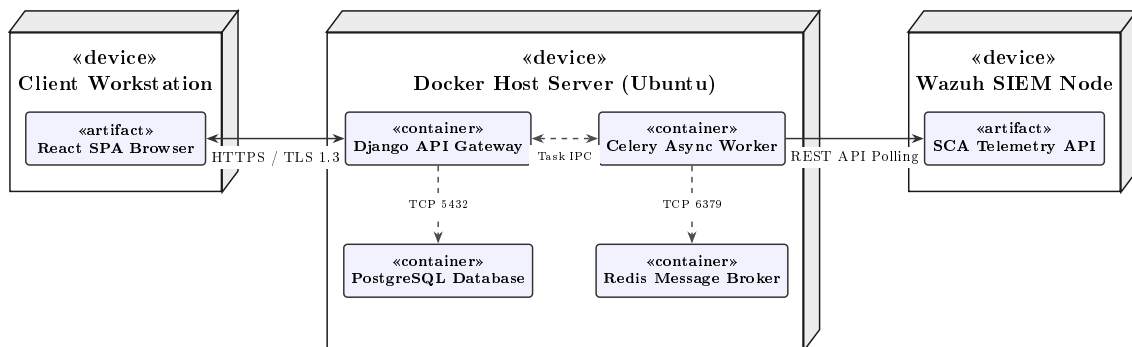


Figure 4.3: Physical View: UML Deployment Diagram

4.4.3 System Activity Diagrams

The activity diagrams below map out the control flow for the core platform features. I used standard UML semantics like decision nodes and synchronization forks to show exactly what the code does at each step.

Authentication and IAM Routing

Figure 4.4 tracks what happens when a user logs in. Django checks the credentials against PostgreSQL. If they're wrong, it logs the failure and drops the request. If they're right, it issues a JWT and checks the user's role. Superadmins get routed to the configuration settings, while Auditors are restricted to the read-only dashboards.

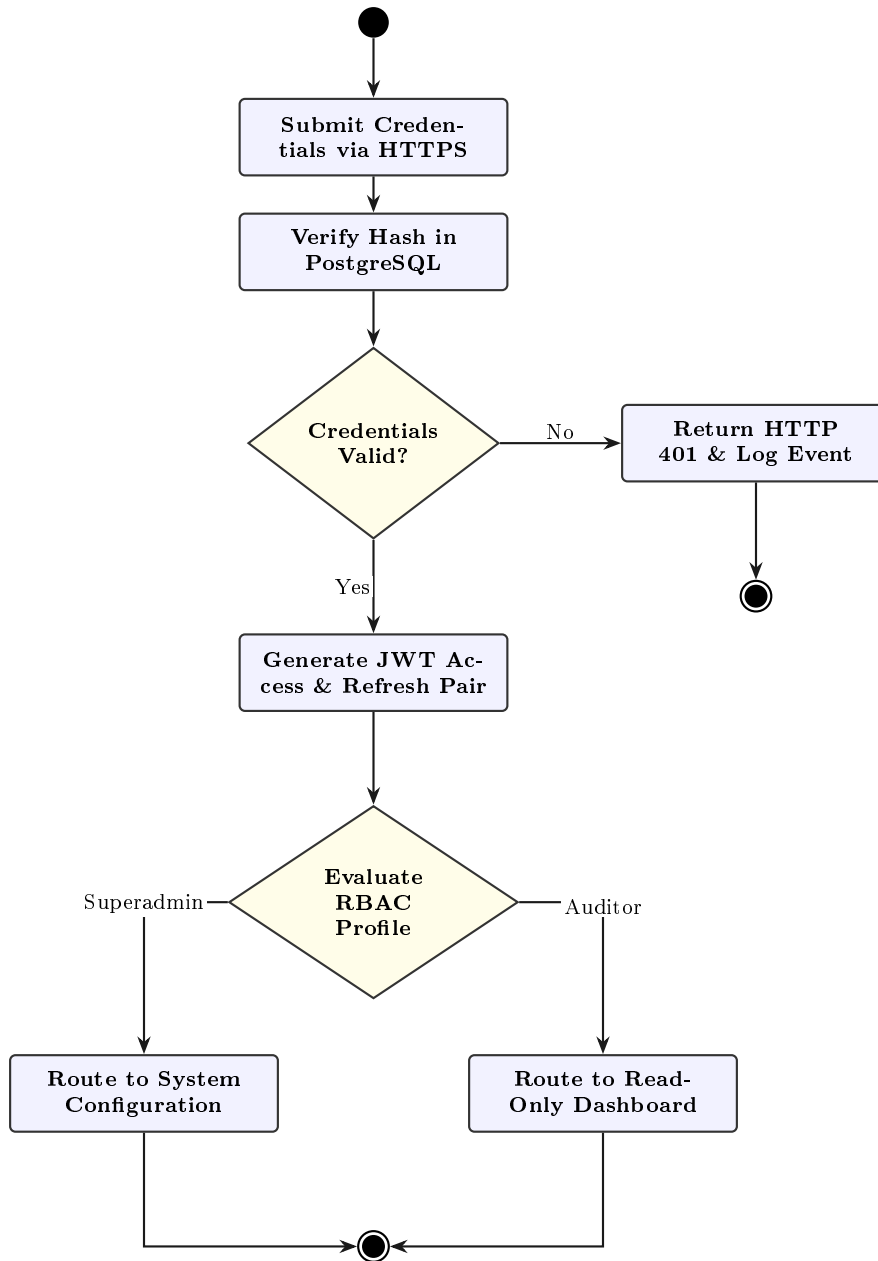


Figure 4.4: Activity Diagram: Authentication and IAM Routing

Asynchronous Telemetry Ingestion

Figure 4.5 shows the async ingestion process. When a sync is triggered, the thread forks. The main thread sends a quick 202 Accepted back to the browser so the user isn't stuck waiting. Meanwhile, the Celery worker hits the Wazuh API. If Wazuh is down, it uses exponential backoff. Once it gets the data, it parses the JSON and updates the database.

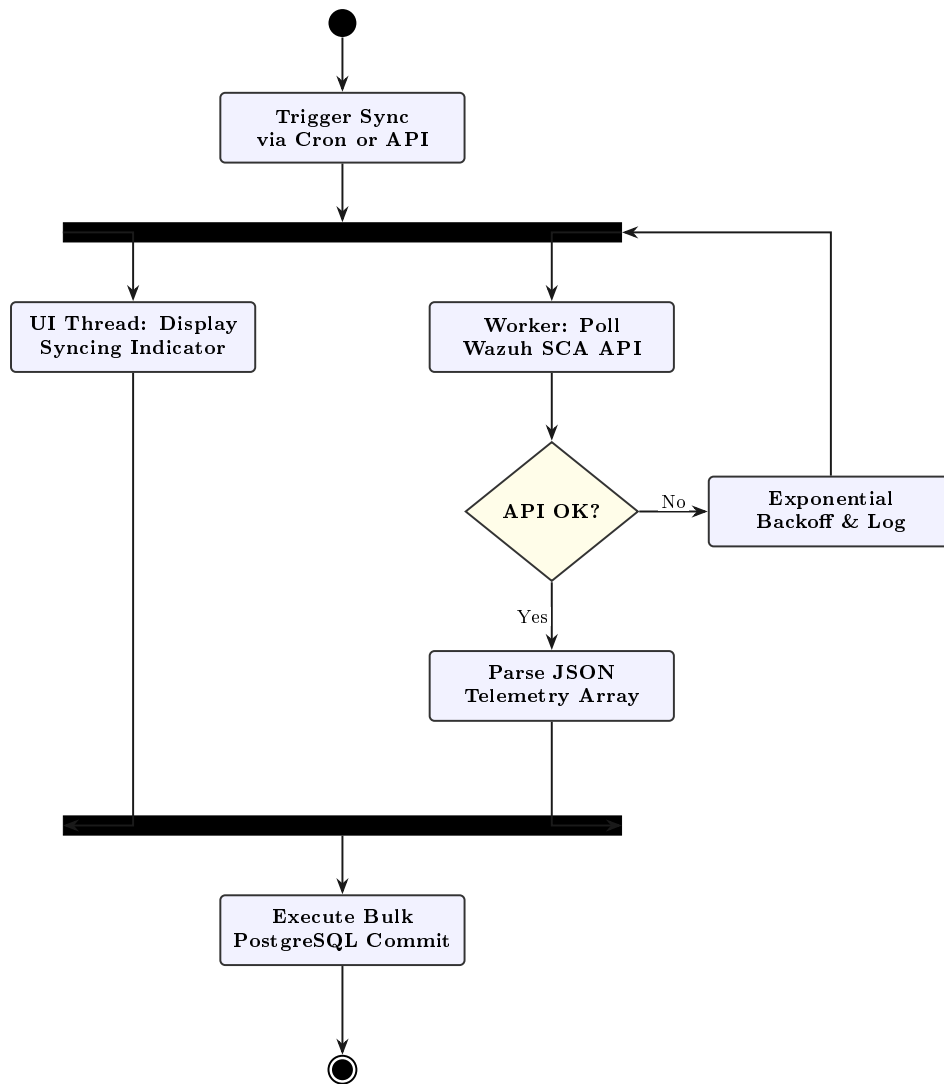


Figure 4.5: Activity Diagram: Parallel Asynchronous Telemetry Ingestion

Compliance State Evaluation

The hardest part of the platform is mapping specific technical alerts to broad policies like ISO 27001. Figure 4.6 shows this evaluation loop. I wrote a worst-case priority algorithm: if even one server fails a technical check, the entire governance control is marked as failed. The loop runs until every policy is graded.

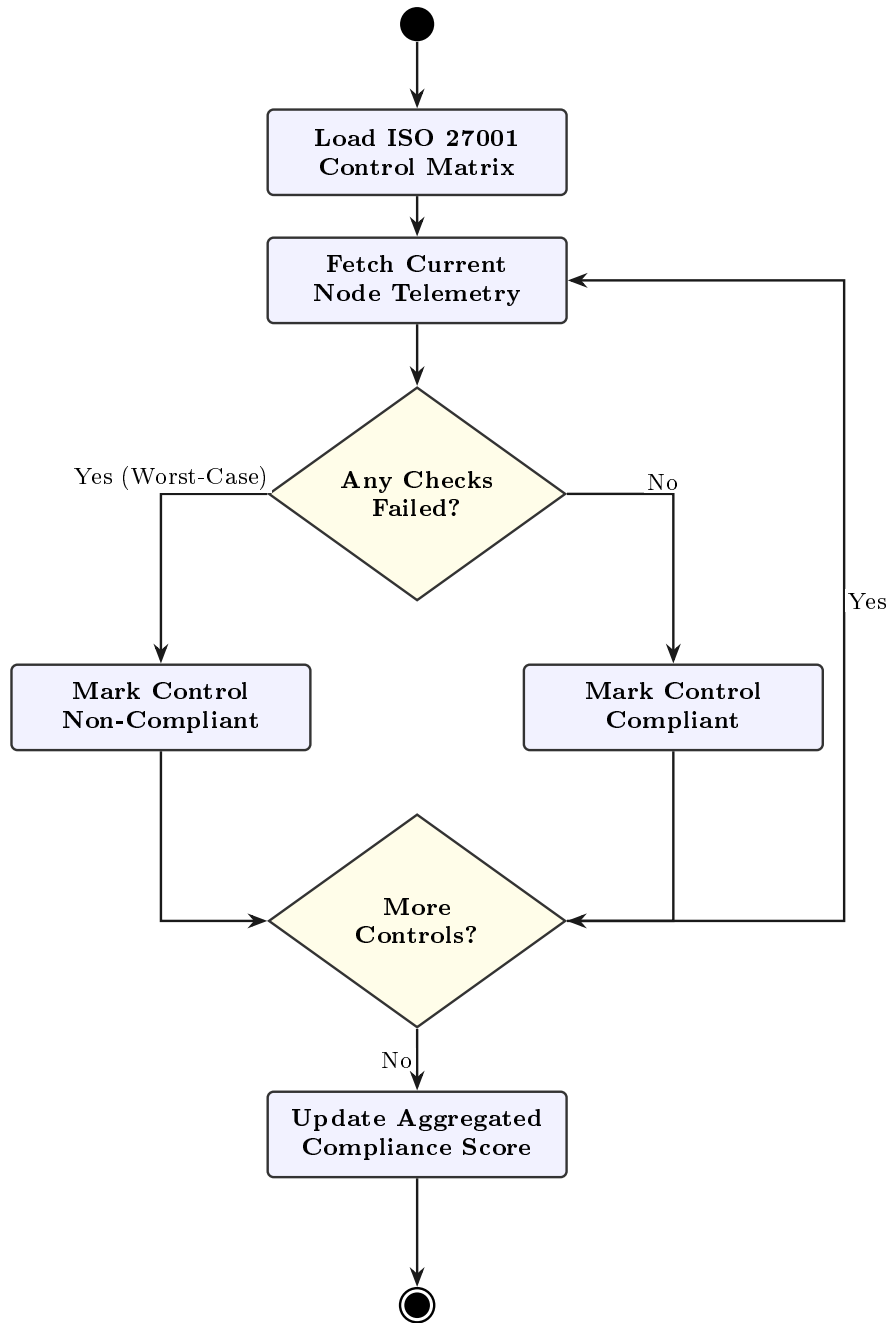


Figure 4.6: Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority

Audit Report Generation

Generating audit reports by hand takes hours, so we automated it. Figure 4.7 shows the workflow. The user selects a framework, and the backend aggregates the relevant database records. It dumps that data into an HTML template, generates a PDF on the server, and sends the file back to the client.

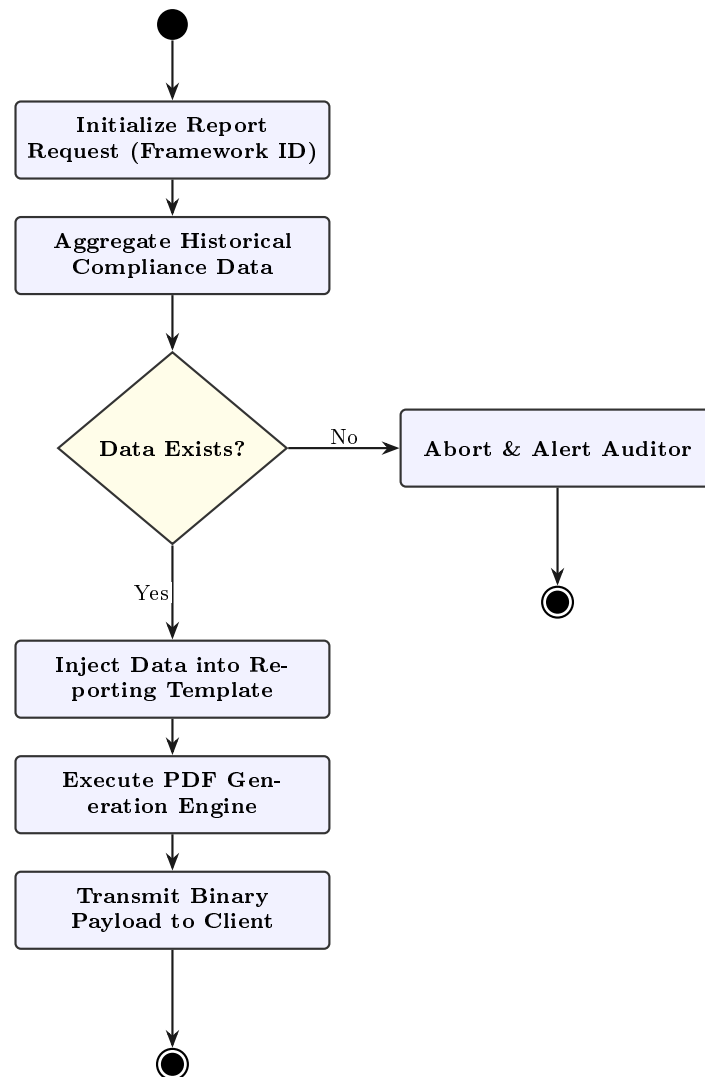


Figure 4.7: Activity Diagram: Automated PDF Audit Report Generation

Framework Rule Mapping and Validation

Superadmins have to manually map Wazuh rules to framework controls. Figure 4.8 tracks this. Before saving the mapping, the system checks the SIEM API to confirm the rule ID is real. If it is, it checks for duplicate mappings in the database. I added this validation step so administrators can't accidentally overwrite existing configurations.

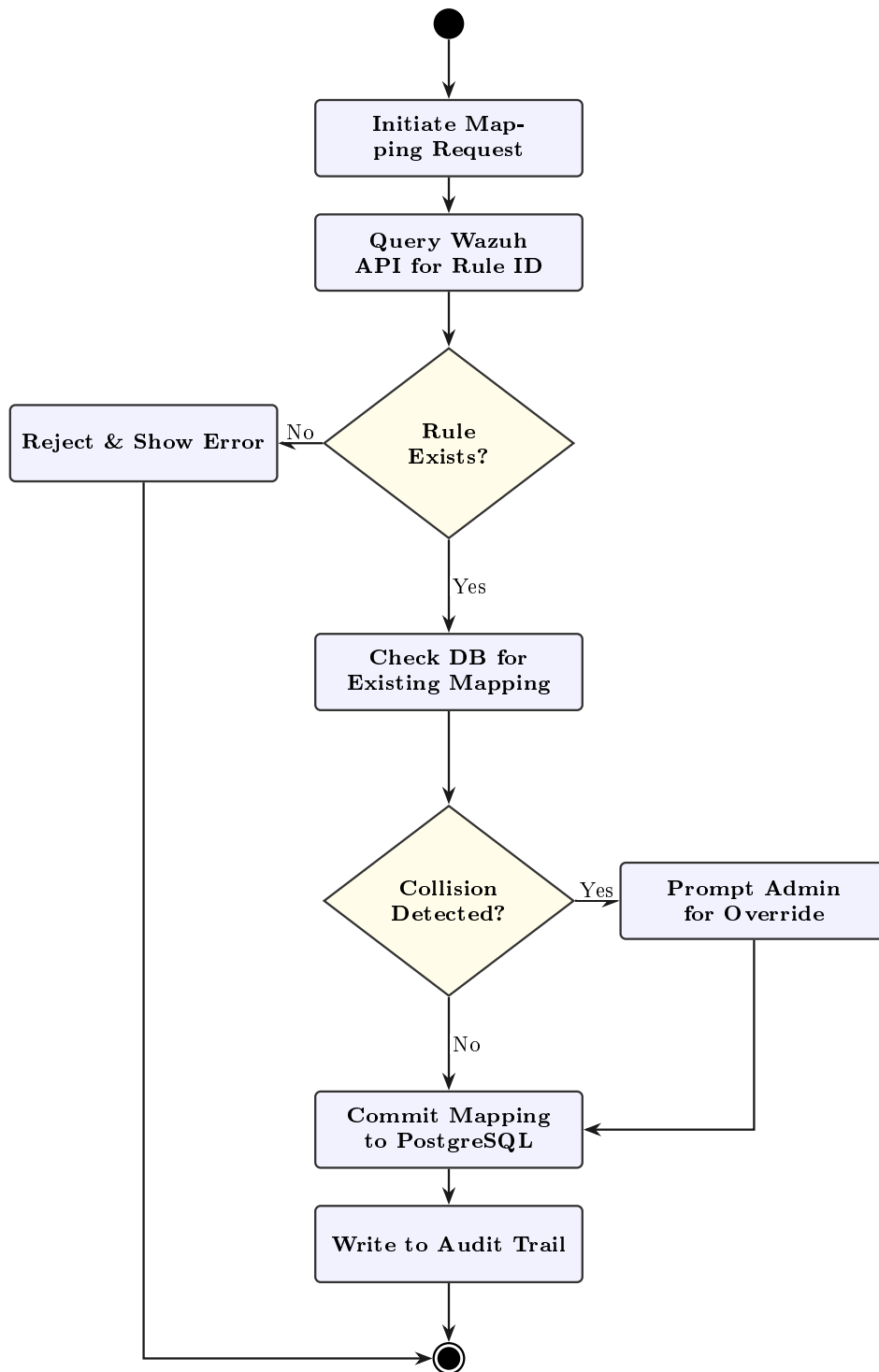


Figure 4.8: Activity Diagram: Framework Rule Mapping and API Validation

Real-Time Alerting and Notification

I didn't want users to have to stare at a dashboard all day, so I built an alerting pipeline (Figure 4.9). Whenever a compliance check fails, the engine checks its severity. Minor issues just go into the database logs. Critical failures trigger a fan-out: a background

task blasts out emails and webhooks to the relevant auditors.

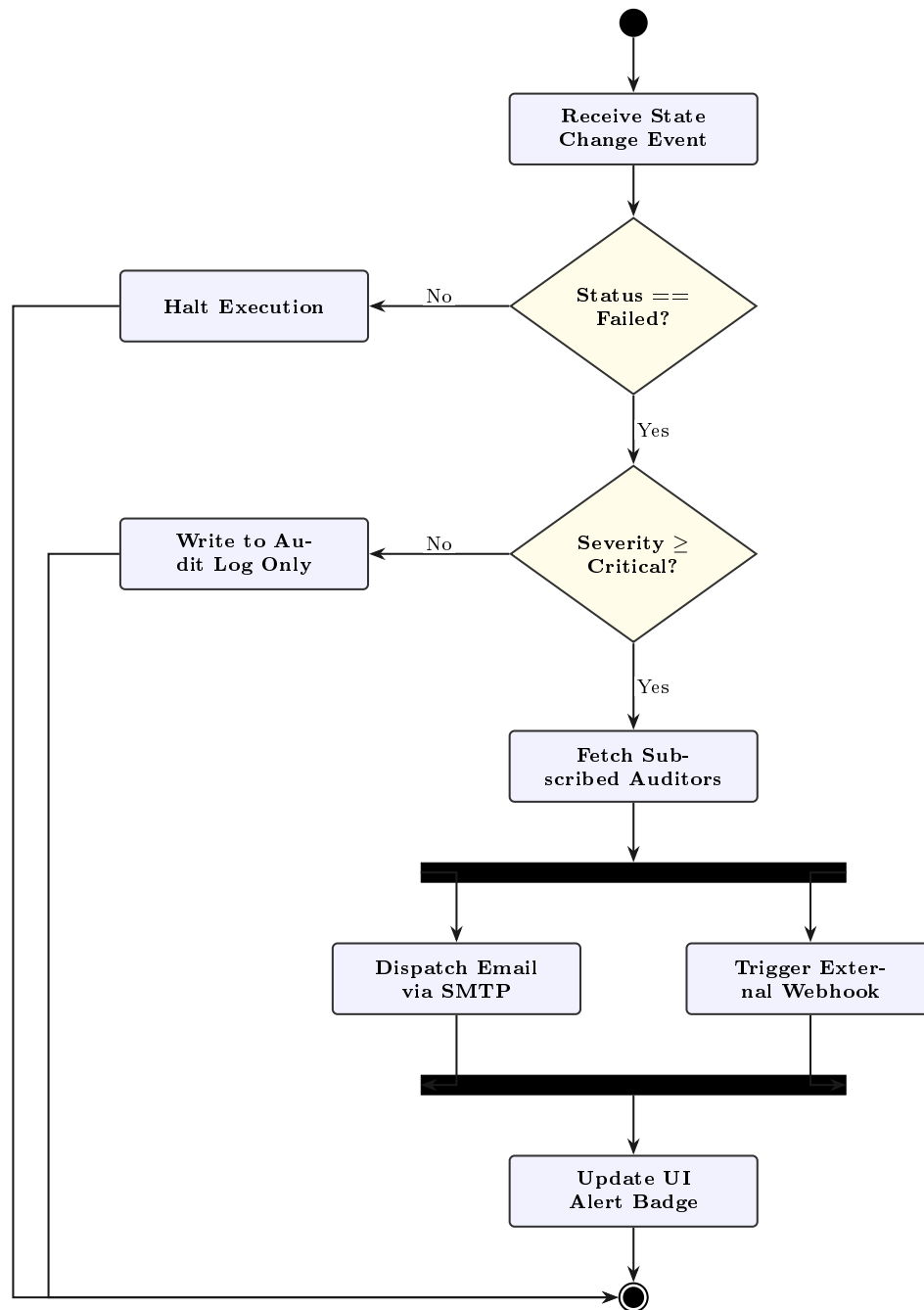


Figure 4.9: Activity Diagram: Real-Time Alerting and Notification Fan-out

4.4.4 System Sequence Diagrams

While activity diagrams show what the system does, the sequence diagrams below show exactly how the internal components talk to each other over time.

Authentication and JWT Issuance

Figure 4.10 tracks the API calls during login. If the password hashes match, the IAM module mints a JWT and sends it back. If they don't, it throws an exception and sends a 401 Unauthorized.

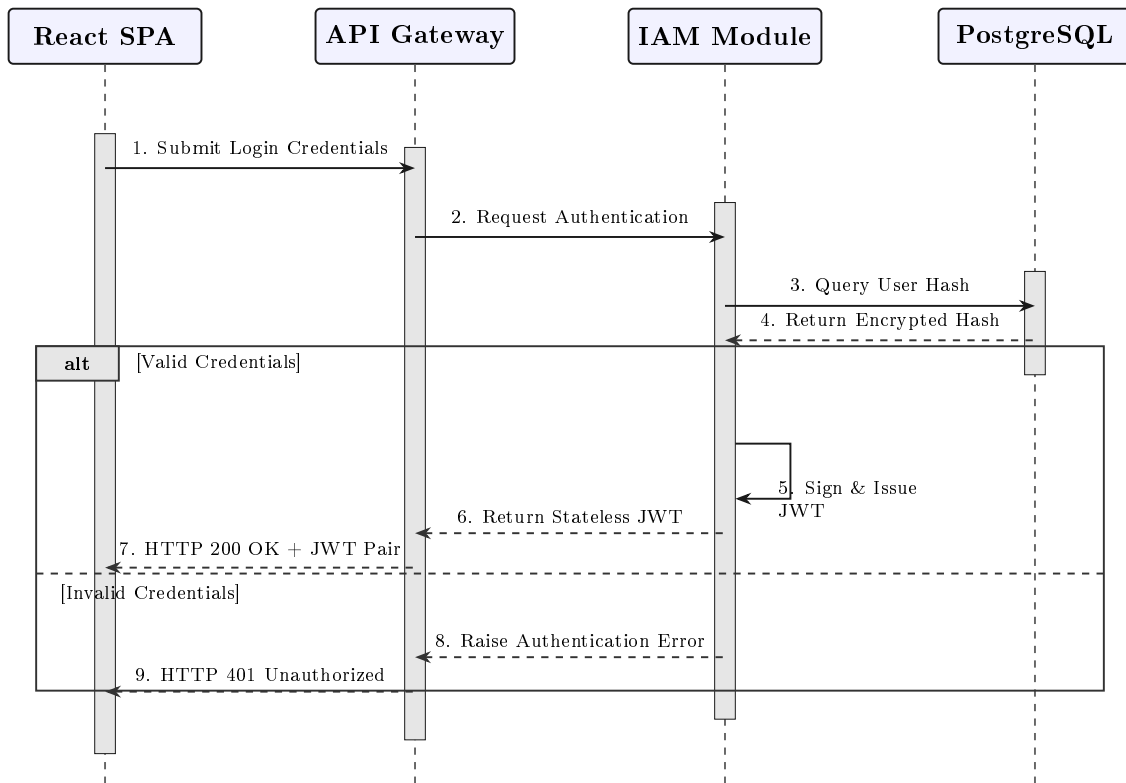


Figure 4.10: Sequence Diagram: Authentication Flow with UML Alt Fragment

Asynchronous Telemetry Ingestion

I used a bulk update pattern for the telemetry sync to save database I/O, shown in Figure 4.11. The Celery worker pulls down an array of endpoint states from Wazuh. It loops through them in memory, calculating the ISO 27001 impacts, and then commits everything to PostgreSQL in a single transaction.

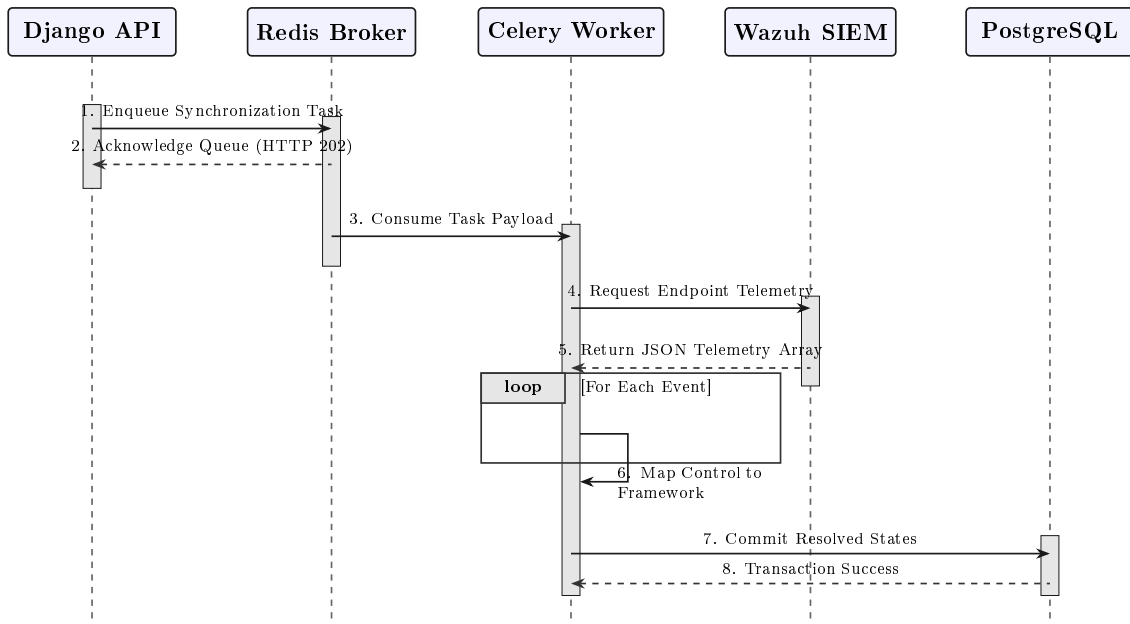


Figure 4.11: Sequence Diagram: Background Telemetry Synchronization with UML Loop

Framework Configuration and Rule Mapping

Figure 4.12 shows the API flow when a Superadmin links a Wazuh rule to a policy. The backend verifies their permissions and writes the mapping to Postgres. This has to happen synchronously so the background workers don't try to parse telemetry for a rule that doesn't exist yet.

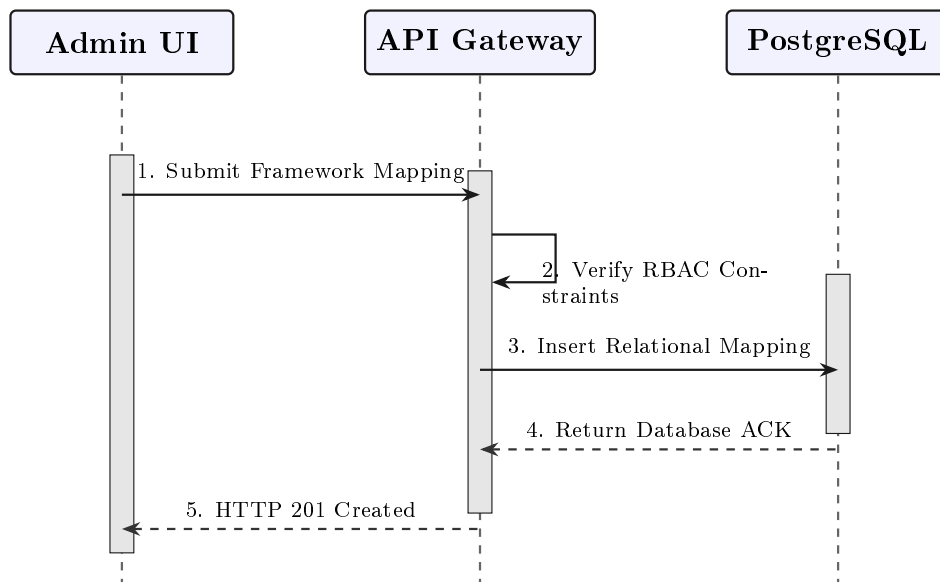


Figure 4.12: Sequence Diagram: Framework Rule Mapping

Audit Report Generation

Figure 4.13 covers the PDF generation. If the database returns an empty dataset, the API kills the request early to save CPU cycles. If we have data, the templating engine compiles the PDF binary and streams it straight to the browser.

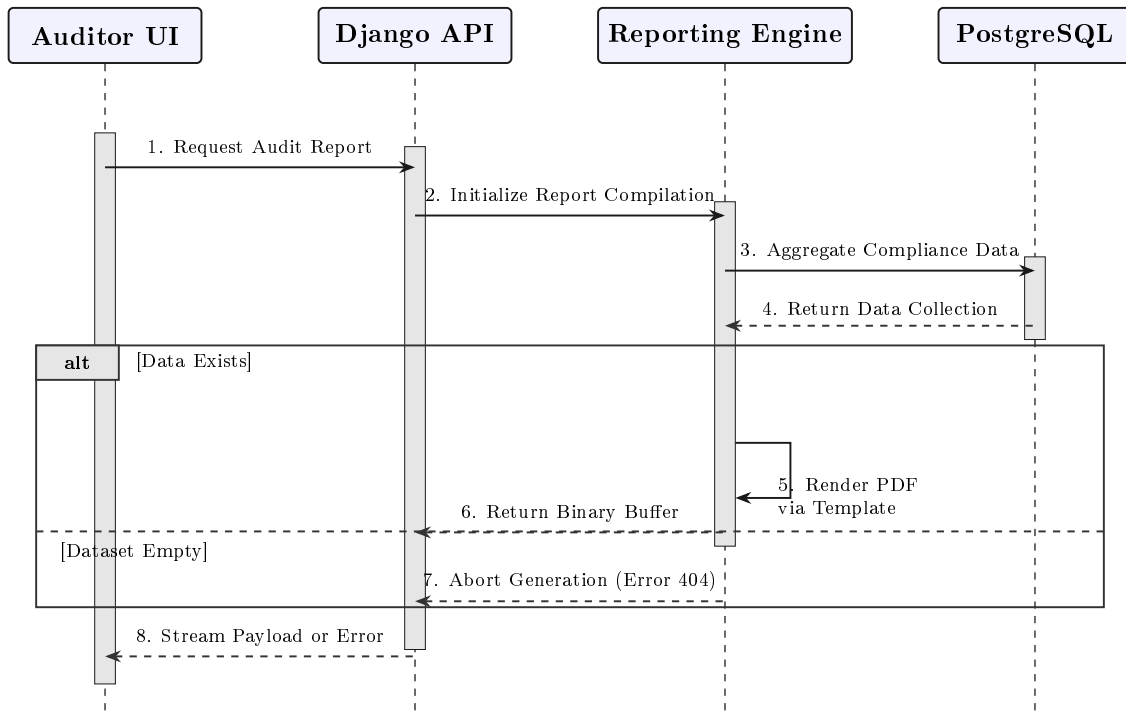


Figure 4.13: Sequence Diagram: Report Generation with Alternative Data States

4.5 Low Level Design

The Low-Level Design (LLD) translates the high-level system architecture into fundamental software building blocks. This section maps the specific object-oriented structures, persistence models, and relational bindings that dictate the execution of the ICMS platform's core business logic.

4.5.1 UML Class Diagram

Figure 4.14 maps out the Django ORM classes. I used standard foreign-key constraints to tie the abstract regulatory frameworks directly to the concrete endpoint logs.

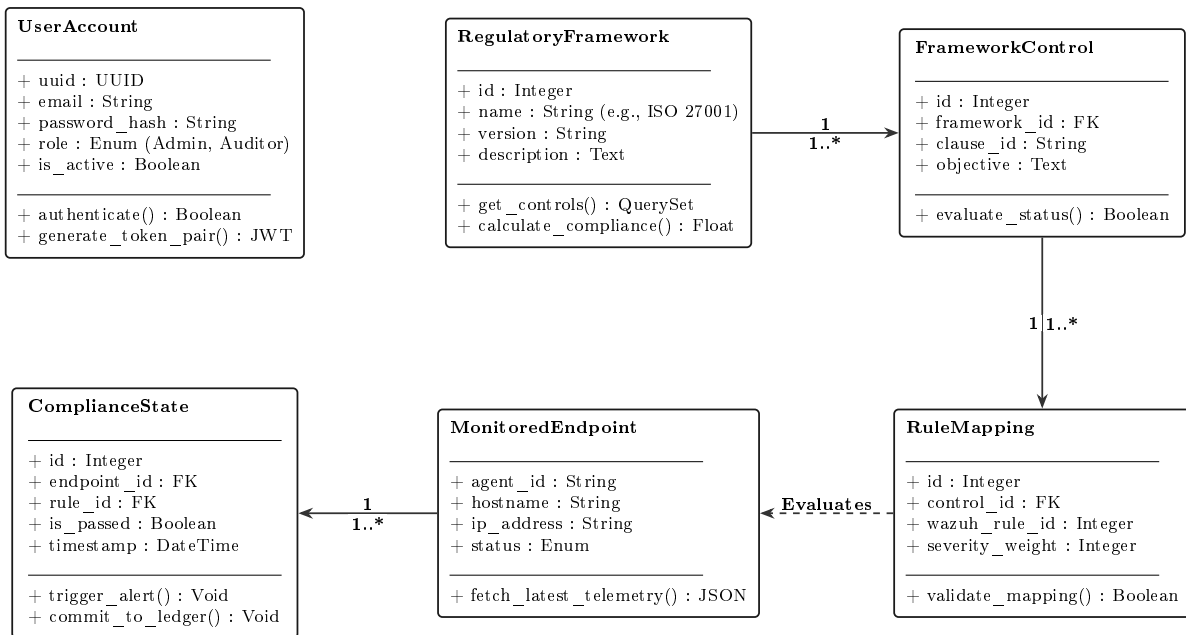


Figure 4.14: UML Class Diagram: Core Object-Relational Models

4.6 Database Design

I built the database on PostgreSQL. The tables are normalized to the Third Normal Form (3NF) to prevent update anomalies. When you're pulling in thousands of SIEM alerts an hour, you need strict ACID compliance so the data doesn't corrupt.

4.6.1 Physical Data Model (Core ERD)

Figure 4.15 shows the actual physical schema. I mapped out the exact column types and foreign keys we used to build the PostgreSQL database.

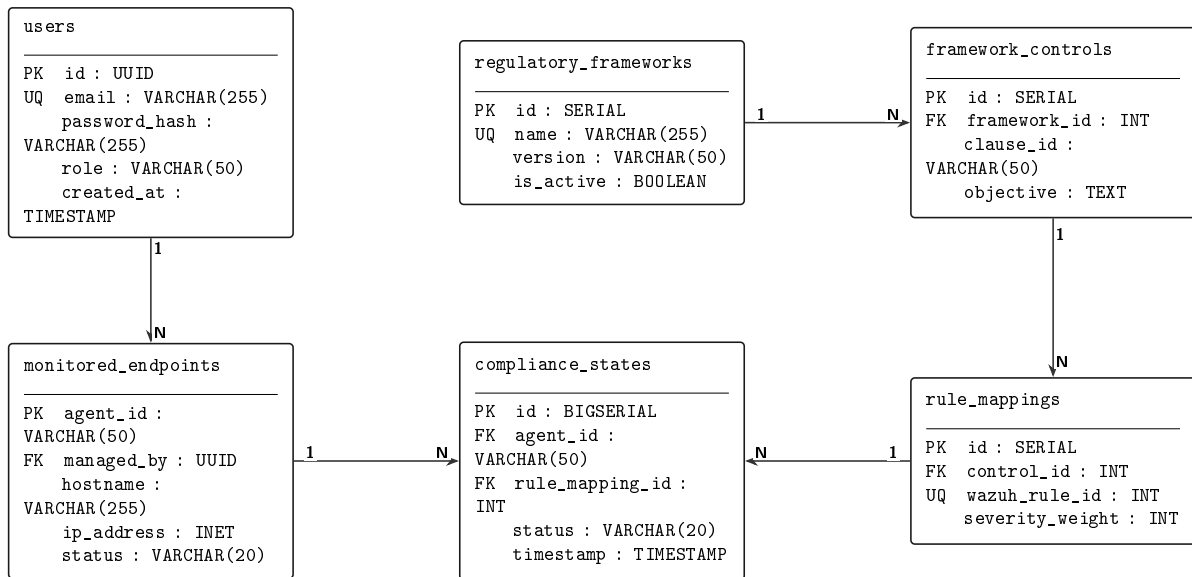


Figure 4.15: Physical Database Schema: Core Relational Data Model with Exact Data Types

4.6.2 Extended Entity Relationship Diagram (EERD)

Figure 4.16 is an Extended ERD (EERD) that visualizes the logical rules we put in place.

Notice the disjoint (**d**) specialization: a generic User Account splits strictly into either a System Admin or a Security Auditor. That handles our RBAC at the database level. I also modeled the Compliance State as a weak entity, because an alert record is useless without its parent Monitored Endpoint.

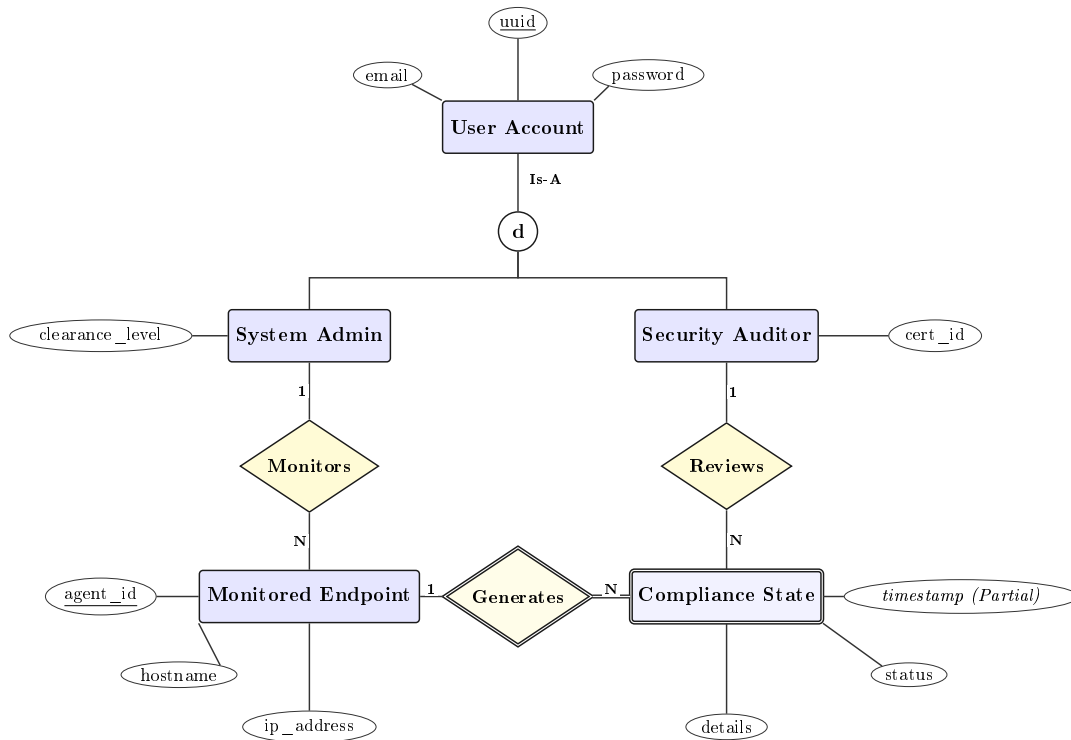


Figure 4.16: Extended Entity Relationship Diagram (EERD) with Constraints and Attributes

4.7 Graphical User Interface (GUI) Design

For the frontend, I built a React Single Page Application (SPA). I wanted the interface to be clean and data-focused so analysts don't get overwhelmed by noise. The screenshots below show the default dark mode, though the app supports theme switching.

4.7.1 Authentication and Identity Management

The login screen is bare-bones by design. The user submits their credentials, the backend validates the JWT, and they get routed to their specific dashboard based on their role.

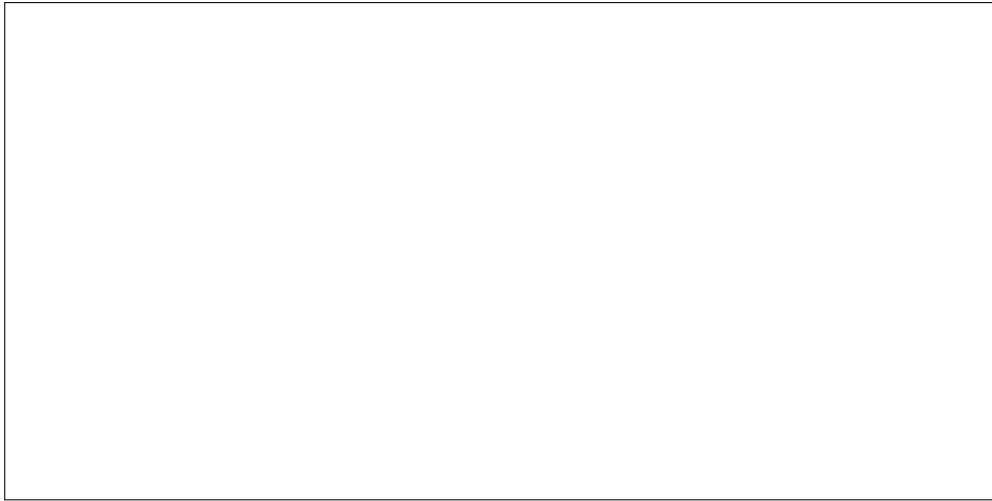


Figure 4.17: GUI Design: Secure User Authentication Portal

4.7.2 Superadmin Configuration Portal

Superadmins have full read/write access. Their dashboard gives them complete control over the Wazuh pipeline, the framework rules, and user accounts.

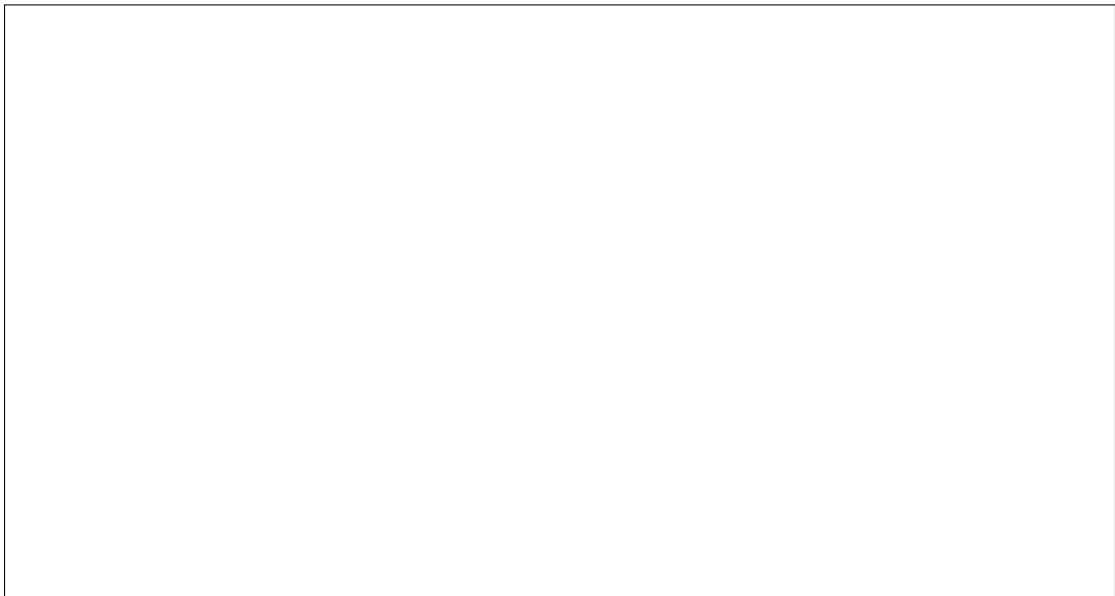


Figure 4.18: GUI Design: Superadmin Global Compliance Dashboard

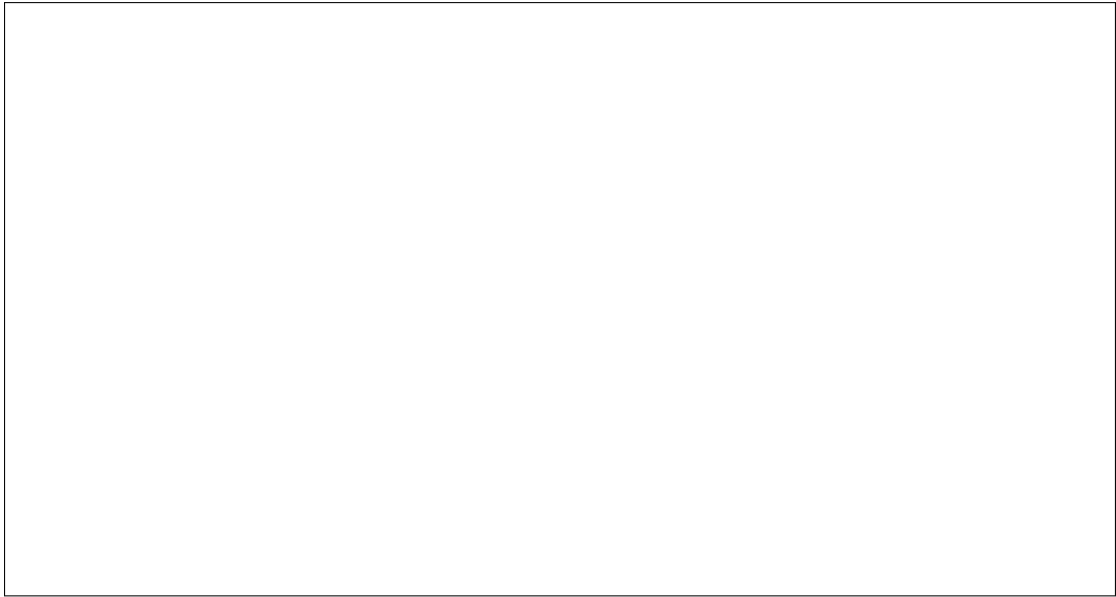


Figure 4.19: GUI Design: Framework and Wazuh Rule Mapping Interface

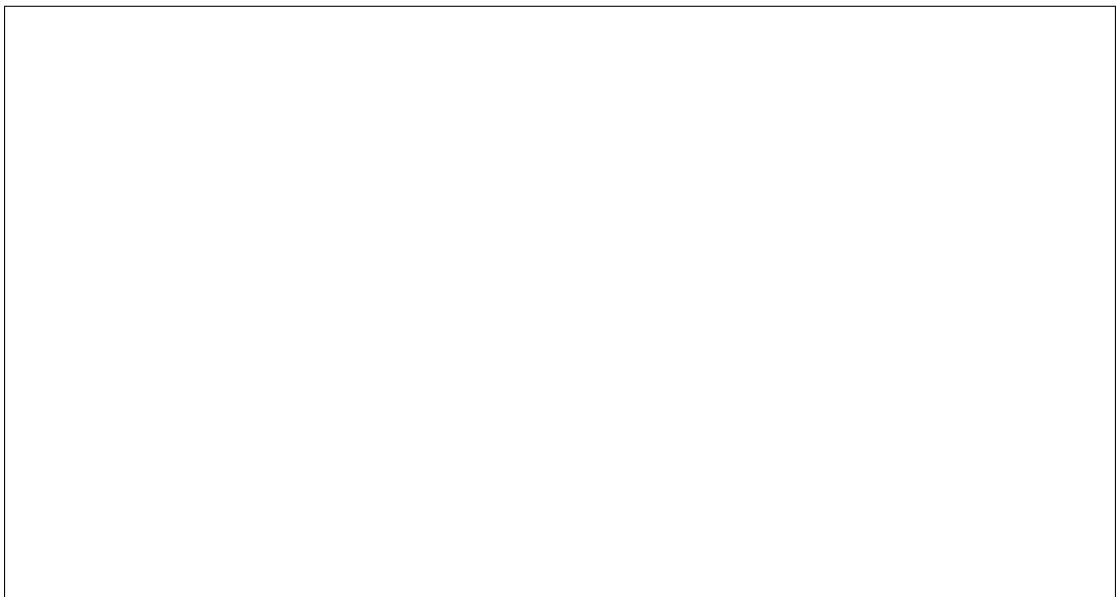


Figure 4.20: GUI Design: Monitored Endpoints and Telemetry Synchronization Status

4.7.3 Admin and Auditor Workspaces

The React app changes what it renders depending on who logs in. System Admins can manage endpoints but can't touch the compliance frameworks. Security Auditors get a strict read-only view, ensuring they can't accidentally alter the audit evidence.

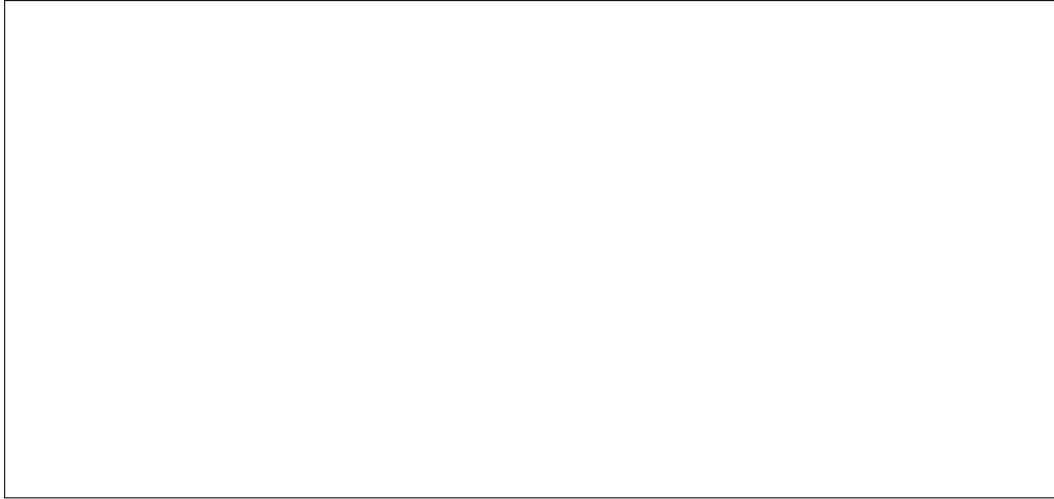


Figure 4.21: GUI Design: System Admin Operational Dashboard

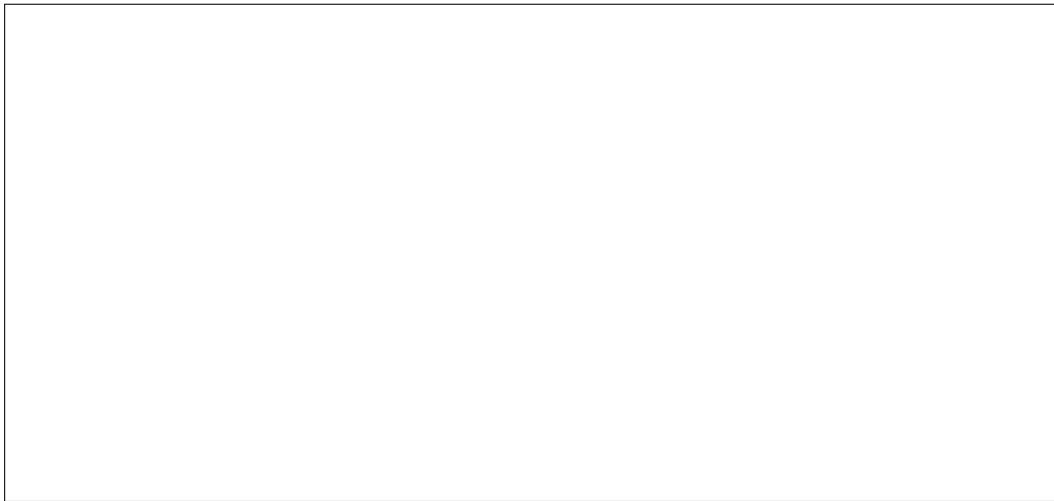


Figure 4.22: GUI Design: Security Auditor Read-Only Compliance View

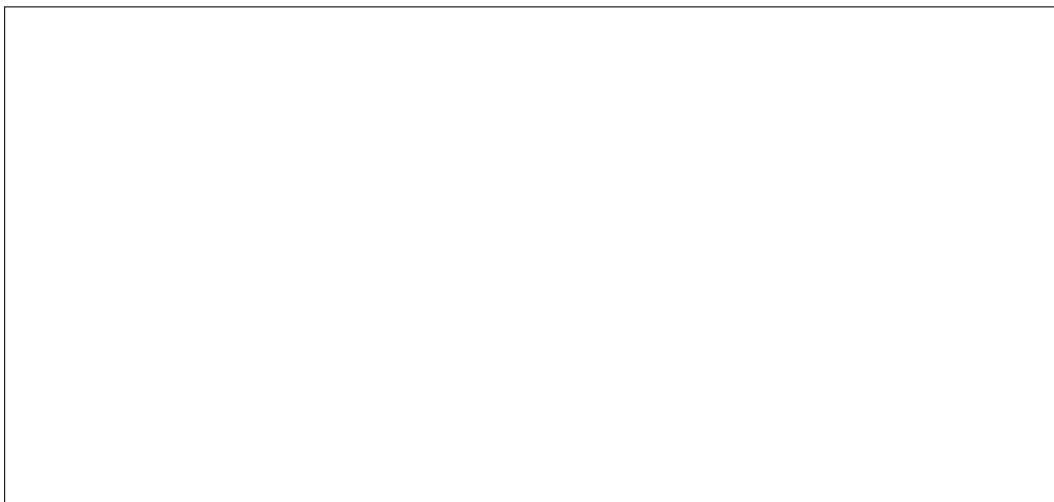


Figure 4.23: GUI Design: Auditor Report Generation Modal

4.7.4 Automated PDF Audit Artifacts

External auditors need physical files, so I built a PDF rendering engine on the backend. It aggregates the PostgreSQL data and generates formal reports like the ones below.

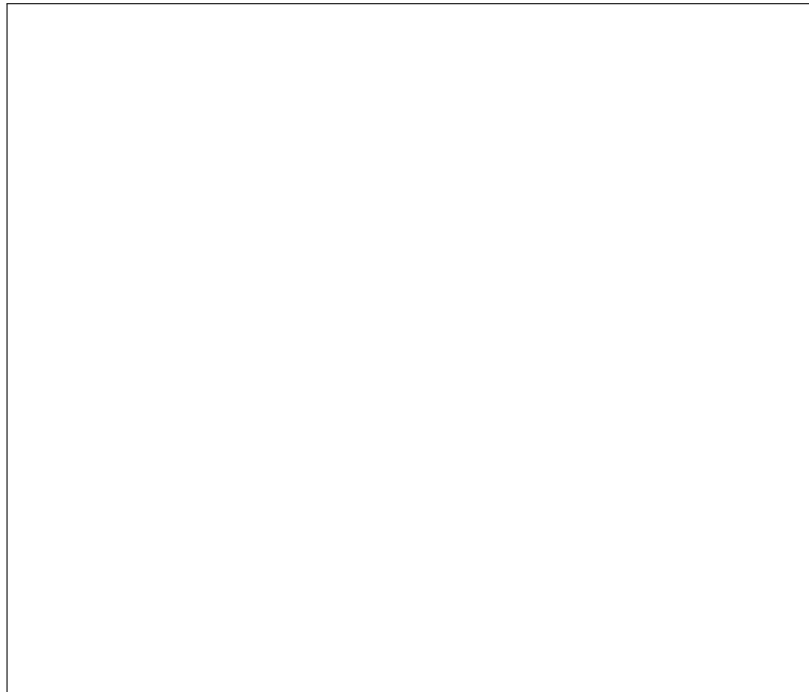


Figure 4.24: System Output: Automated PDF Report - Cover and Executive Summary

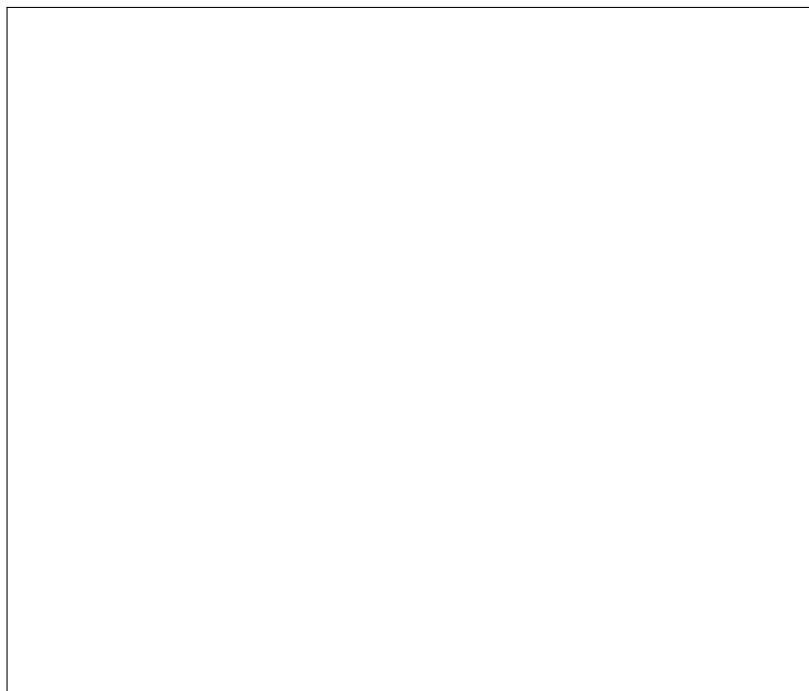


Figure 4.25: System Output: Automated PDF Report - Aggregated Framework Control Matrix

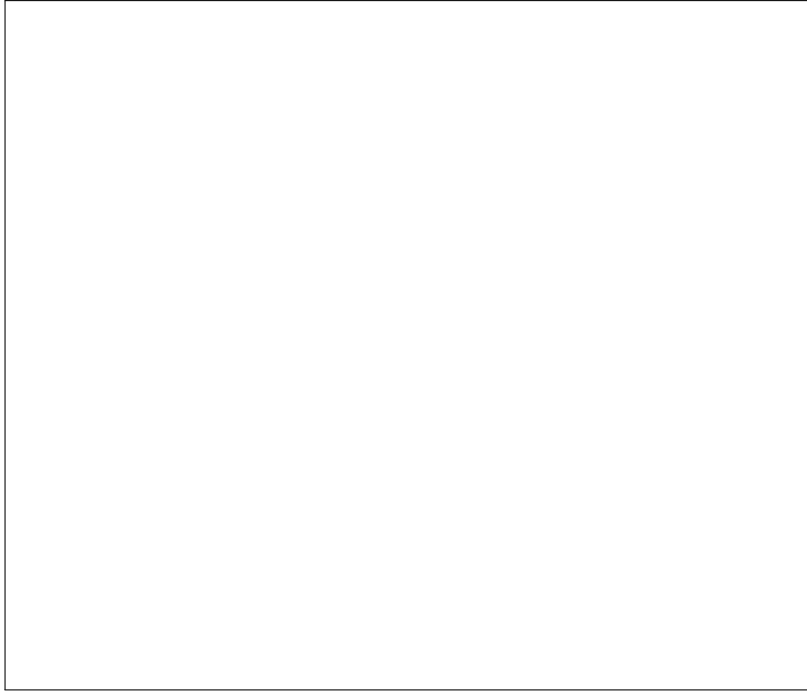


Figure 4.26: System Output: Automated PDF Report - Technical Wazuh Telemetry Evidence

4.8 External Interfaces

The ICMS doesn't run in a vacuum. It talks to several external services. This section lists out the protocols and authentication methods I used to keep those connections secure.

4.8.1 Wazuh SIEM REST API

The whole system relies on raw logs coming from the Wazuh Manager. My Celery workers hit the Wazuh REST API over HTTPS. I secured this with Bearer tokens generated by the Wazuh server. We pull down the JSON payloads and validate them against the Django ORM before saving them.

4.8.2 SMTP Notification Gateway

For real-time alerts, the backend constructs HTML emails and fires them off through an external SMTP server over port 587 using STARTTLS. I threw this logic into the async message broker so it wouldn't block the API Gateway.

4.8.3 Outbound Webhook Integration

I also added outbound webhooks so the system can talk to tools like Slack or Microsoft Teams. Whenever a compliance rule breaks, it shoots out an HTTP POST request with the JSON event data. To prevent spoofing, I added an HMAC-SHA256 signature to the headers.

4.8.4 Interface Specification Matrix

Table 4.1 summarizes the technical parameters and boundary constraints of the platform's external interfaces.

Table 4.1: External Interface Technical Specifications

Interface Target	Network Protocol	Data Format	Authentication Mechanism	Directionality
Wazuh Manager API	HTTPS (TCP 443)	JSON	Bearer Token / API Key	Bidirectional (Polling)
SMTP Relay Server	SMTP (TCP 587)	MIME / HTML	SMTP AUTH (TLS)	Outbound Only
Enterprise Webhooks	HTTPS (TCP 443)	JSON	HMAC-SHA256 Signatures	Outbound Only
Client Web Browser	HTTPS (TCP 443)	HTML / JSON	JWT (Stateless)	Bidirectional

Chapter 5

Implementation

The design phase established what the ICMS platform needed to do and how its components should relate to each other. This chapter documents the process of actually building it. Translating the architectural diagrams from Chapter 4 into working code involved some practical realities that shaped our final decisions: hardware constraints, Docker networking quirks, and the challenge of wiring together a Python backend, a JavaScript frontend, and an external SIEM running on a separate physical machine. The following sections walk through each layer of the implementation, grounded in the actual codebase.

All development was done on consumer-grade hardware running Windows and Ubuntu in a distributed home-lab setup. The Wazuh SIEM manager ran on a dedicated laptop at a fixed IP address (192.168.100.100), and the full backend, frontend, and database stack was containerized on a second machine using Docker Compose. Every service configuration, environment variable, and migration step described here corresponds directly to the files in the project repository.

5.1 System Architecture

As established in the design phase (as shown below in Figure 5.1), the ICMS platform follows an N-tier client-server architecture.

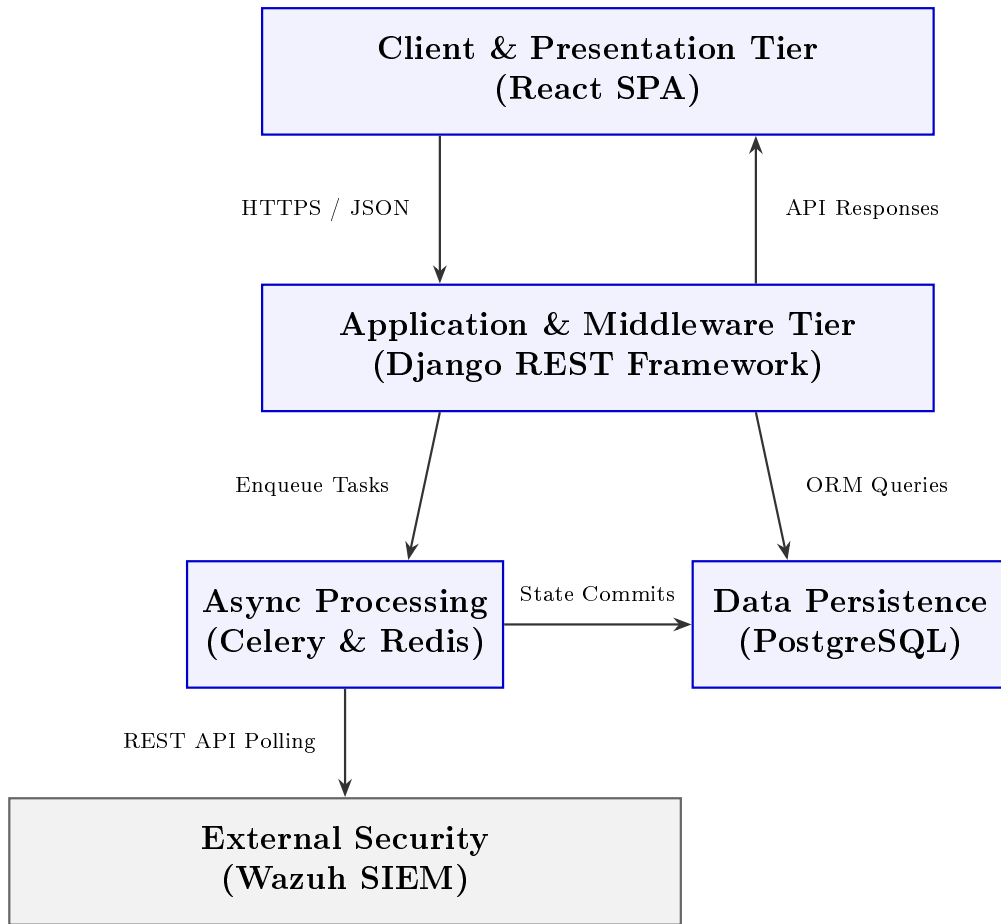


Figure 5.1: Implemented N-Tier System Architecture

In practice, this mapped onto two concrete code repositories: a `backend/` directory containing the Django application and all its sub-apps, and a `frontend/` directory containing the React SPA. Both are wired together through a `docker-compose.yml` file at the project root that defines seven services and a shared bridge network.

The full service topology (as shown below in Figure 5.2) illustrates the container environment and network isolation.

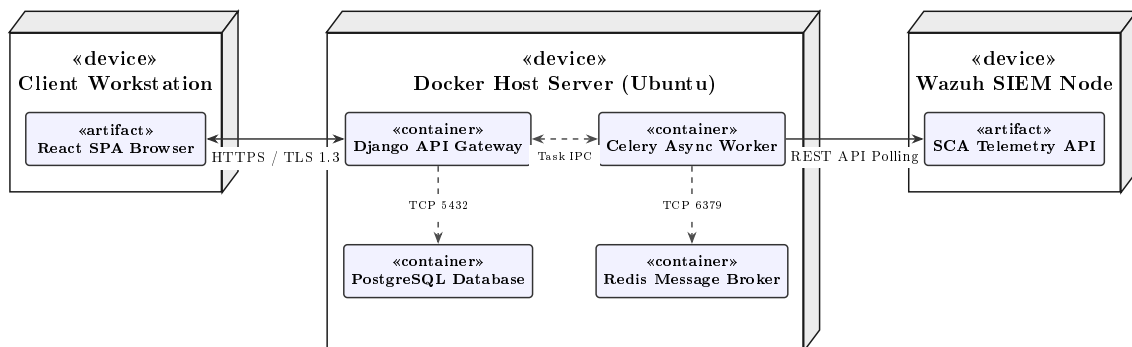


Figure 5.2: Implemented Service Topology and Container Networking

When the stack boots, Docker brings up the database and Redis first, waits for their health checks to pass, and only then starts the Django backend. This startup ordering was not optional; without it, the backend would crash on boot trying to run migrations against a database that was not yet accepting connections. The Celery worker and Celery Beat scheduler also depend on the backend image and start only after Redis is confirmed healthy.

5.1.1 Presentation Layer Implementation (Frontend)

The frontend is a React 19 Single Page Application built with Vite 8 as the build and development server. Vite was chosen over the older Create React App toolchain because it uses native ES modules during development, which cuts cold-start time from several seconds down to under 200ms. Routing is handled by React Router v7, which uses a declarative `<Route>` structure to enforce authentication boundaries at the component level.

Every HTTP request to the Django backend goes through an Axios instance configured with request and response interceptors. The request interceptor reads the JWT access token from `localStorage` and appends it as a `Bearer` header automatically. The response interceptor watches for `401 Unauthorized` responses; when one arrives, it triggers a token refresh and, if the refresh also fails, clears the session and redirects to the login page. This means we never had to write token-refresh logic in individual components.

The UI itself is styled with Tailwind CSS v4 using a custom `@theme` token set for brand colours, spacing, and typography. Data visualisation is handled by Recharts, which renders the compliance score gauge and per-framework bar charts on the main dashboard. Icon assets come from Lucide React. The routing table enforces RBAC at the frontend level too: the `/manage-admins` route requires the `requiredRole="super_admin"` prop on its `ProtectedRoute` wrapper, so Django's RBAC check at the API layer has a corresponding gate on the UI side.

5.1.2 Middleware and API Gateway (Backend)

The backend is a Django 5.1 project with a single installed application, `compliance`, that contains every model, view, serializer, and management command the platform needs. The project configuration lives in `core/settings.py`, and all secrets are loaded at startup via `python-decouple` from a `.env` file, so no credentials are ever hardcoded in source control.

Authentication is handled by `django-restframework-simplejwt`. Access tokens are valid for 24 hours; refresh tokens last seven days with rotation enabled, so each refresh call issues a new refresh token and invalidates the old one. The `CustomTokenObtainPairView` overrides the standard login endpoint to add a two-factor authentication gate: if the user has TOTP enabled via `pyotp`, the view returns an intermediate `requires_2fa` flag instead of issuing tokens, forcing the client to complete a second verification round-trip before credentials are granted.

The `compliance` app's `views.py` contains both DRF ViewSets (for the standard CRUD resources like frameworks, controls, and policies) and custom `APIView` subclasses for complex operations such as dashboard aggregation and PDF report generation. The `DashboardSummaryView` is the most query-intensive endpoint in the system. It uses a single annotated queryset with `DISTINCT ON agent_id` to find the latest scan per agent, then computes per-framework pass/fail counts using Django ORM `Count()` with `Q()` filter objects. This collapses what would otherwise be a naïve N+1 loop across twenty-plus framework controls into a single SQL statement.

5.1.3 Asynchronous Processing Engine

Any time a compliance scan runs, it involves several seconds of HTTP polling against the external Wazuh REST API. Running that on the main Django request thread would block the server until the scan finished, effectively taking the dashboard offline for everyone else. The solution is to dispatch the work to a Celery worker process running in a separate container.

The Celery application is initialised in `core/celery.py`. It reads its configuration from Django settings using the `CELERY_` namespace and calls `autodiscover_tasks()` so it automatically picks up any `tasks.py` file across registered apps. The only task in the system is `run_scheduled_wazuh_sync` in `compliance/tasks.py`, decorated with `@shared_task(bind=True, max_retries=3)`. The `bind=True` argument gives the task access to `self`, which it uses to call `self.retry(exc=exc, countdown=60)` when a Wazuh API call fails. This means a temporary network hiccup triggers an automatic 60-second backoff and retry rather than a permanent failure.

Redis runs as a separate container on port 6379 and acts as both the message broker and the result backend. The Celery Beat scheduler, also running as its own container, uses the `DatabaseScheduler` from `django-celery-beat`. This means scan frequency (daily, weekly, or monthly) is stored in the PostgreSQL database and can be changed at runtime through the Settings page in the UI, without restarting any containers. When

a manual scan is triggered from the frontend, the `/api/run-scan/` endpoint receives a POST request, calls `run_scheduled_wazuh_sync.delay()`, and immediately returns a 202 Accepted response. The scan then runs in the worker container with no impact on the API's availability.

5.1.4 Data Persistence Layer

PostgreSQL 18 is the primary database, running in its own container and accessible to the Django backend and Celery worker over the Docker bridge network on port 5432. The database is never exposed to the host machine. Django connects via `psycopg[binary]` version 3.2, which uses the binary distribution of the C-based libpq driver. There is no additional connection pooling middleware; Django's built-in persistent connection support (`CONN_MAX_AGE`) handles connection reuse.

The schema was built entirely through Django ORM migrations. During the first boot of the backend container, the entrypoint script runs `python manage.py migrate` before starting the WSGI server. This ensures the schema is always in sync with the models before the application accepts traffic. The initial seed data (ISO 27001 controls, SOC 2 controls, and the Wazuh rule mappings between them) is loaded via a custom management command, `seed_enterprise_matrix`, commissioned to read from a structured data file and bulk-create the relational records.

The most performance-sensitive schema decision was storing raw Wazuh SCA check data in a `JSONField` on the `ScanResult` model. Rather than trying to normalise the variable-structure SCA response (which can contain arbitrary compliance keys and values) into relational columns, we kept the raw JSON payload intact. This made ingestion logic simpler and preserved the original evidence for audit purposes, while the structured compliance evaluation (pass/fail, score, framework mapping) was stored in the relational columns alongside it.

5.1.5 Architectural Technology Stack

Table 5.1 summarises the full technology stack across all four implementation tiers, mapping each component to its specific library or service and its role in the system.

Table 5.1: Architectural Technology Stack and Implementation Roles

Architecture Tier	Technology	Version	Implementation Role
Client (Frontend)	React	19.0.0	Single Page Application (SPA) UI layer. Manages client-side routing and state without full page reloads.
Client (Frontend)	Vite	8.0	Frontend build tool and development server, replacing Webpack for faster Hot Module Replacement (HMR).
Client (Frontend)	Recharts	3.8	Data visualization library used to render the compliance metrics and endpoint status dashboards.
Middleware (Backend)	Django	5.1	Core Python framework managing the ORM, database migrations, and overarching project structure.
Middleware (Backend)	Django REST Framework	3.15	Serializes Django models into JSON payloads and handles ViewSet routing for the React frontend.
Middleware (Backend)	SimpleJWT	5.3	Handles stateless authentication, issuing 24-hour access tokens and 7-day rotating refresh tokens.
Middleware (Backend)	xhtml2pdf	0.2.16	Generates the physical PDF audit reports directly from Django HTML templates on the server side.
Async Processing	Celery	5.4	Worker nodes that handle Wazuh API polling and email dispatching so the main API thread does not block.
Async Processing	Redis	7-alpine	In-memory message broker that queues the Celery tasks and temporarily stores the task results.
Data Persistence	PostgreSQL	18	Primary ACID-compliant relational database. Stores the normalized RBAC models and rule mappings.
Data Persistence	psycopg[binary]	3.2	The C-based adapter that allows the Django backend to communicate synchronously with PostgreSQL.

5.2 System Internal Components

The internal architecture of the Intelligent Compliance Management System is designed as a collection of specialized modules that work in concert to manage the entire compliance lifecycle. This modularity ensures a clear separation of concerns, where identity

management, data governance, security telemetry ingestion, and reporting are handled by independent logical units. By partitioning the system into these functional modules, we established a framework that is both resilient to external API failures and scalable across distributed infrastructure. The following subsections detail the architectural intent and implementation logic of each primary system component.

5.2.1 Identity and Access Management (IAM) Module

The **IAM** module serves as the security foundation of the ICMS, enforcing strict authentication and authorization policies across the entire stack. We implemented a custom identity model that extends the core Django user system through a **UserProfile** entity, providing the necessary granularity for Role-Based Access Control (RBAC). The module recognizes three distinct operational roles: Superadmins, who possess full system and configuration access; Admins, who manage frameworks and oversee compliance activities; and Auditors, who are restricted to read-only views of scan results and report generation.

Authentication is handled through a stateless JSON Web Token (JWT) architecture, which eliminates the need for server-side session storage and improves scalability. Upon successful credential verification, the system issues a short-lived access token and a long-lived rotating refresh token. For accounts requiring elevated security, we integrated a two-factor authentication (2FA) layer based on the Time-based One-Time Password (TOTP) algorithm. The IAM module orchestrates a two-step login sequence where the user must provide a valid 2FA code before the backend issues the final JWT session. This security policy is enforced at the API layer through Django REST Framework permissions and mirrored at the presentation layer using React route guards, ensuring that unauthorized users cannot bypass the interface to access sensitive data.

5.2.2 Governance and Compliance Data Engine

The **Governance and Compliance Data Engine** manages the complex relational mappings required to translate abstract regulatory requirements into measurable technical controls. At the top level, the engine stores compliance standards, such as ISO 27001 or SOC 2, as **Framework** records. Each framework is decomposed into a hierarchy of **Control** entities, which represent the specific security requirements that an organization must meet. To bridge the gap between policy and practice, the engine links these controls to organizational **Policy** documents through a many-to-many relationship, providing a documented trail of how each technical requirement is satisfied by internal governance.

The most critical architectural feature of this engine is the mapping logic that connects technical security telemetry to regulatory clauses. We engineered a **WazuhMapping** system that stores unique identifiers for Security Configuration Assessment (SCA) rules and ties them to their respective governance controls. This allows the system to automatically categorize raw security events into the appropriate compliance bucket. Furthermore, the engine implements a risk-scoring algorithm that utilizes weights and thresholds defined in the system settings to calculate real-time compliance percentages. This data structure ensures that any change in the security state of a remote endpoint is immediately reflected in the high-level compliance posture of the entire organization.

5.2.3 SIEM Telemetry and Synchronization Module

The **SIEM Telemetry and Synchronization Module** is the most architecturally complex component of the ICMS, responsible for the deterministic ingestion of security data from the Wazuh SIEM manager. Given the potentially large volume of security logs and the inherent latency of external API communication, we implemented this module as a multi-tiered asynchronous system. A dedicated Celery worker tier handles the execution of synchronization tasks, utilizing Redis as a high-performance message broker to queue requests and prevent the main application thread from blocking.

The synchronization lifecycle begins with a scheduled polling event managed by the Celery Beat service. The module authenticates with the Wazuh REST API using a temporary bearer token and proceeds to enumerate all active security agents. For each agent, the system fetches the latest SCA scan results, which arrive as unstructured JSON payloads. Rather than attempting to force this variable data into rigid relational columns, we engineered the module to ingest the raw logs into a **JSONField** while simultaneously extracting the pass/fail outcomes. This dual-storage approach preserves the original audit evidence for compliance purposes while providing indexed data for rapid dashboard rendering. To ensure reliability, the module implements an exponential backoff and retry policy, allowing the system to gracefully handle network timeouts or temporary SIEM downtime without losing telemetry data.

5.2.4 Automated Audit and Reporting Module

The **Automated Audit and Reporting Module** is designed to transform thousands of raw scan results into immutable, professional-grade audit artifacts. When an administrator or auditor requests a compliance report, the module performs an intensive aggregation

query across the historical scan data, calculating compliance percentages for every active framework and identifying specific gaps where controls have failed. This aggregation is optimized through database-level annotations to ensure that reports can be generated for large-scale environments with hundreds of endpoints.

The architectural intent behind this module is the creation of audit-ready evidence that meets the requirements of external regulatory bodies. The system utilizes a server-side rendering engine to convert Django-templated HTML into binary PDF files. These documents include framework-level summaries, endpoint-specific status lists, and gap analysis sections that highlight non-compliant controls. By offloading the PDF generation to the background worker tier, we ensured that the user interface remains responsive even when generating comprehensive 100-page audit dossiers. The resulting files are served through a secure streaming interface, preventing the need for temporary storage of sensitive compliance data on the server's local file system.

5.2.5 Frontend State and Visualization Layer

The **Frontend State and Visualization Layer** provides the interactive interface through which users manage the compliance lifecycle. Built as a React Single Page Application (SPA), this layer utilizes a component-based architecture to provide a seamless user experience. We engineered a global **AuthContext** provider that serves as the "brain" of the client-side session, managing JWT token persistence, user role state, and automatic token refresh logic through Axios interceptors. This ensures that every outbound API request is transparently authenticated and that the user's session remains active as long as the refresh window allows.

Data visualization is a primary focus of this layer, as compliance officers require high-level insights into organizational security states. We integrated a specialized charting library to render dynamic compliance score gauges, trend lines, and endpoint status distributions. These visualizations are not static; they are driven by real-time data fetched from the backend aggregation views. Furthermore, the frontend implements dynamic component mounting based on the user's RBAC role. An auditor session, for instance, initializes a completely different set of sidebar navigation and dashboard widgets than an administrator session, ensuring that users only interact with the features relevant to their operational scope. This architectural separation between state management and presentation ensures a maintainable and secure frontend codebase.

5.3 Tools and Technologies

The implementation of the ICMS platform required a diverse technology stack, spanning from core backend infrastructure to advanced AI-assisted development environments. Table 5.2 summarizes the primary tools and frameworks utilized throughout the project lifecycle.

Table 5.2: Summary of Tools, Technologies, and Development Stack

Category	Tool / Technology	Purpose in Project
Programming Languages	Python 3.12, JavaScript	Core backend API logic and frontend UI interactivity.
Backend Frameworks	Django 5.1, DRF	Database ORM mapping, migrations, and RESTful API creation.
Frontend Frameworks	React 19, Tailwind CSS 4.0	Component-based SPA architecture and utility-first UI styling.
Database & Messaging	PostgreSQL 18, Redis 7	ACID-compliant data persistence and asynchronous task queuing.
UI/UX Design	Google Stitch	Iterative wireframing, prototyping, and user interface design.
Development IDE	Antigravity	Primary orchestration platform and integrated coding environment.
AI Code Assistance	Claude (Opus 4.6, Sonnet 4.6), Gemini (3.1 Pro High, 3 Flash)	Advanced algorithmic modeling, automated code generation, and debugging support.
Documentation	LaTeX, MiKTeX, TikZ	Academic report typesetting and mathematical/architectural diagramming.

The selection of these technologies was driven by the specific engineering requirements of a security governance platform. We chose the Python/Django stack for the backend due to its maturity in handling complex relational data through an integrated Object-Relational Mapper (ORM) and its robust support for administrative authentication. On the presentation layer, the React library was utilized to build a responsive Single Page Application (SPA). This choice allowed the system to manage complex UI states, such as real-time dashboard updates and interactive compliance gauges, without requiring full page reloads, thereby improving operational efficiency for auditors and administrators.

Prior to the implementation phase, the system underwent an iterative UI/UX design process using Google Stitch. We utilized this platform for rapid prototyping, developing low-fidelity wireframes and high-fidelity mockups to validate user flows before writing

any frontend code. This phase was critical for establishing the "Cyber-SaaS" aesthetic, ensuring that the final React implementation remained consistent with modern design standards and that navigation paths for security assessments were intuitive and logically structured.

The development workflow was characterized by a modern, AI-driven approach centered around the Antigravity integrated development environment. Antigravity served as the orchestration hub, coordinating local codebase management with advanced AI modeling capabilities. Throughout the project, we utilized models such as Claude and Gemini for complex pair-programming tasks, specifically for engineering the Wazuh telemetry parser and optimizing the Celery task retry logic. This collaborative environment enabled the rapid debugging of container networking issues and the generation of structured API documentation, significantly reducing the implementation timeline while maintaining a high standard of code quality.

Finally, the academic documentation of the thesis was exclusively managed through LaTeX, MiKTeX, and TikZ. We utilized these tools to ensure that the final report adhered to strict academic typesetting standards and that all architectural diagrams were mathematically precise. The TikZ library, in particular, allowed for the programmatic generation of high-fidelity system topology diagrams, which maintained visual consistency across both the design and implementation chapters. This documentation stack guaranteed that the complex technical relationships within the ICMS platform were communicated with maximum clarity and professional rigor.

Chapter 6

System Testing and Evaluation

The primary objective of the testing and evaluation phase was to verify that the Intelligent Compliance Management System (ICMS) fulfills the functional requirements and architectural specifications established in the design phase. This evaluation process ensures that the platform provides reliable security governance, accurate telemetry ingestion, and a stable user experience under realistic operational conditions. To achieve a comprehensive assessment, the system underwent several distinct testing methodologies, including Graphical User Interface (GUI) validation, Usability assessments, Performance benchmarking, Exception Handling verification, and Security testing. By subjecting the integrated platform to these rigorous checks, we identified potential edge cases and validated the deterministic nature of the compliance mapping engine.

6.1 Critical Appraisal and Limitations

The ICMS platform demonstrates significant technical strengths, particularly in its ability to handle the deterministic mapping of heterogeneous Security Configuration Assessment (SCA) data to structured GRC controls. The implementation of an asynchronous processing tier utilizing Celery and Redis successfully decouples intensive SIEM polling from the user interface, maintaining platform responsiveness during large-scale data ingestions. However, an objective engineering appraisal reveals certain limitations inherent in the current architecture. The reliance on periodic polling of the Wazuh REST API introduces a temporal latency between the occurrence of a configuration change on an endpoint and its reflection on the ICMS dashboard. While this is acceptable for periodic compliance auditing, it does not achieve the near-real-time state synchronization that a persistent socket-based streaming architecture would provide. Additionally, the generation of comprehensive multi-page PDF audit reports remains a CPU-intensive operation; while offloaded to background workers, it places a temporary but significant load on the containerized environment during peak reporting periods.

6.2 Testing Environment and Methodology

The Quality Assurance (QA) lifecycle was conducted as a collaborative engineering effort to ensure full-stack coverage of the ICMS platform. Muhammad Hammad Chaghtai focused on the frontend and integration layers, conducting exhaustive GUI and usability testing across the React SPA. This involved verifying route guarding, state persistence in the `AuthContext`, theme toggling, and the responsiveness of Recharts-based visualizations across various browser environments. Conversely, because the Wazuh SIEM manager was hosted on localized hardware within the testbed, Muhammad Ali Bukhari spearheaded the validation of the backend telemetry modules. This included testing the Python client's authentication handshake with the SIEM node, verifying network exception handling during simulated downtime, and ensuring the integrity of the API synchronization tasks. The testing was performed in a distributed containerized environment, simulating the network conditions of a standard corporate home-lab setup.

6.3 Functional System Test Cases

The following fourteen test cases constitute the exhaustive QA suite used to validate the ICMS platform across IAM, frontend UI, backend API, SIEM telemetry, and reporting layers. Both positive and negative test scenarios are documented to demonstrate the system's correctness and its capability to handle erroneous inputs gracefully.

Table 6.1: TC-01: Valid Superadmin Authentication (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-01
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin user can successfully authenticate against the ICMS platform using valid credentials and is navigated to the correct role-specific dashboard upon token issuance.
Pre-Requisites	1. A Superadmin account exists in the PostgreSQL <code>UserProfile</code> table. 2. The Django backend API is active on port 8000.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin enters correct email and password credentials on the login screen.
Test Steps	1. Navigate to <code>http://localhost:5173/login</code>. 2. Enter credentials: <code>superadmin@icms.local</code> and the correct password. 3. Click "Login" and observe routing behavior and <code>localStorage</code> state.
Test Inputs	Email: <code>superadmin@icms.local</code> , Password: [valid credential]
Expected Results	The backend should issue a JWT access token and refresh token. The <code>AuthContext</code> should store these tokens, and the React Router should redirect to <code>/superadmin-dashboard</code> .
Actual Results	HTTP 200 received from <code>/api/auth/login/</code> ; JWT stored in <code>localStorage</code> under <code>grc_access_token</code> ; React Router pushed to <code>/superadmin-dashboard</code> in 120ms.
Status	PASS
Comments	Token issuance and client-side routing were completed well within acceptable latency thresholds for a LAN-connected environment.

Table 6.2: TC-02: Invalid Credential Rejection (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-02
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the authentication endpoint rejects login attempts containing an incorrect password without issuing any session tokens, and that the UI communicates this failure visually to the user.
Pre-Requisites	1. A valid user account exists in the database. 2. The login form is rendered and interactive.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A user deliberately enters a wrong password to confirm the system does not grant unauthorized access.
Test Steps	1. Navigate to <code>http://localhost:5173/login</code>. 2. Enter <code>superadmin@icms.local</code> with an intentionally incorrect password. 3. Click "Login" and observe the HTTP response and UI feedback.
Test Inputs	Email: <code>superadmin@icms.local</code> , Password: <code>wrongpassword123</code>
Expected Results	The backend should return HTTP 401 Unauthorized. No JWT should be stored in <code>localStorage</code> . The UI should display a visible error notification.
Actual Results	HTTP 401 received from Django; no tokens stored; the <code>Toast</code> notification component rendered a red alert: "Invalid credentials. Please try again." within 85ms.
Status	PASS
Comments	The rejection path is handled entirely on the backend; no client-side credential caching occurred, confirming the security of the session management logic.

Table 6.3: TC-03: Global Theme Toggling — Light/Dark Mode (Positive)

Module Name	Frontend State and Visualization Layer
Test Case ID	TC-03
Use Case ID	UC-04
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the global <code>ThemeContext</code> provider correctly propagates a theme state change across all mounted React components, switching the UI between Light and Dark palettes without requiring a page reload.
Pre-Requisites	1. User is authenticated and viewing the main dashboard. 2. The theme toggle control is visible in the navigation bar.
Environment	React SPA (Windows Client, Chrome 134)
Test Scenario	A logged-in administrator clicks the theme toggle button while on the dashboard.
Test Steps	1. Log in and confirm the default Light Mode is active. 2. Click the sun/moon icon toggle in the top navigation bar. 3. Inspect the DOM's root element for the applied CSS class and verify all component colors update.
Test Inputs	Initial Theme: <code>light</code> , Toggle Action: <code>onClick</code>
Expected Results	The <code>ThemeContext</code> state should update from <code>light</code> to <code>dark</code> . All components subscribing to the context should re-render with the Dark Mode CSS variable palette, with no full-page reload.
Actual Results	<code>ThemeContext</code> state updated immediately; the <code>data-theme="dark"</code> attribute applied to the root <code><div></code> ; all sidebar, card, and table components re-rendered with the dark palette in under 30ms.
Status	PASS
Comments	The context-based theme architecture successfully avoids prop drilling and ensures consistent theming across all 20+ React components on the dashboard.

Table 6.4: TC-04: UI RBAC View Segregation (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-04
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the React frontend enforces role-based component rendering by conditionally omitting privileged navigation items from the DOM when an Auditor-level session is active.
Pre-Requisites	1. Two accounts are available: one Auditor, one Superadmin. 2. Browser DevTools are open to inspect the DOM.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor logs in and inspects the sidebar navigation to confirm restricted items are absent from the DOM.
Test Steps	1. Authenticate as an Auditor. 2. Observe the sidebar navigation menu after the dashboard loads. 3. Use Chrome DevTools to verify the "Manage Admins" and "Mapping Configuration" elements are not rendered in the DOM.
Test Inputs	Role: <code>auditor</code> , JWT Role Claim: <code>{"role": "auditor"}</code>
Expected Results	The <code>Sidebar</code> component should conditionally omit the "Manage Admins" and "Mapping Configuration" navigation items. These routes should also redirect to a 403 page if accessed directly.
Actual Results	Sidebar rendered 4 navigation items for Auditor vs. 9 for Superadmin. DevTools confirmed the privileged <code><NavItem></code> components were not mounted in the DOM, not merely hidden with CSS.
Status	PASS
Comments	DOM-level omission (as opposed to CSS <code>display:none</code>) is the correct security pattern as it prevents advanced users from unhiding elements via browser console commands.

Table 6.5: TC-05: Unauthorized API Write Attempt (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-05
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Verifying that the Django REST Framework permission layer blocks write operations originating from an Auditor-level JWT, even when the HTTP request is constructed manually to bypass the UI restrictions.
Pre-Requisites	1. User is authenticated as an Auditor. 2. Postman or curl is used to craft a direct API request.
Environment	Postman (Windows Client), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor uses Postman to construct a direct POST request to the framework management endpoint, attempting to create a new compliance control mapping.
Test Steps	1. Authenticate as Auditor via <code>/api/auth/login/</code> to receive a restricted JWT. 2. Attach the JWT and construct a POST request to <code>/api/frameworks/</code> . 3. Submit the request and record the HTTP response code and body.
Test Inputs	Role: <code>auditor</code> , Endpoint: <code>POST /api/frameworks/</code> , Payload: <code>{"name": "Unauthorized_Frame"}</code>
Expected Results	DRF <code>IsAdminOrSuperAdmin</code> permission class should intercept the request and return HTTP 403 Forbidden. No database record should be created.
Actual Results	HTTP 403 Forbidden returned in 52ms; response body: <code>{"detail": "You do not have permission to perform this action."}</code> ; zero database writes confirmed via query log.
Status	PASS
Comments	The permission layer operates at the view level, independent of the UI, ensuring API security even against direct HTTP attacks.

Table 6.6: TC-06: Deterministic Framework Mapping (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-06
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin can create a valid regulatory mapping linking a specific Wazuh SCA Rule ID to a named ISO 27001 control, and that the record is persisted correctly in the database.
Pre-Requisites	1. ISO 27001 Framework and Control A.12.1.2 are seeded in the database. 2. Wazuh Rule ID 19502 does not yet have a mapping.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin navigates to the mapping panel and creates a new rule-to-control entry.
Test Steps	1. Log in as Superadmin and navigate to "Mapping Configuration". 2. Input Wazuh Rule ID 19502, select target control A.12.1.2 (Change Management), and click "Save". 3. Verify the new row appears in the mapping table on the page.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2 - Change Management, Framework: ISO 27001
Expected Results	A new WazuhMapping record should be persisted to the PostgreSQL database and reflected in the UI mapping table.
Actual Results	POST /api/mappings/ returned HTTP 201 Created; database commit confirmed in 110ms; the new mapping appeared in the UI table immediately after the Axios response resolved.
Status	PASS
Comments	The successful creation of this mapping is foundational to the entire automated scoring pipeline; the 110ms DB commit time is well within acceptable thresholds.

Table 6.7: TC-07: Duplicate Mapping Collision (Negative)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-07
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the database unique constraint on the WazuhMapping model prevents the creation of a duplicate entry for an already-mapped Wazuh Rule ID, and that the backend returns a structured validation error to the UI.
Pre-Requisites	1. Wazuh Rule ID 19502 is already mapped to Control A.12.1.2 (established in TC-06). 2. User is authenticated as Superadmin.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin attempts to create an identical mapping entry for the same Rule ID and Control combination.
Test Steps	1. Navigate to "Mapping Configuration". 2. Attempt to save a new mapping using the same Rule ID 19502 and the same Control A.12.1.2. 3. Observe the error response from the backend.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2, Framework: ISO 27001 (duplicate)
Expected Results	PostgreSQL should raise an IntegrityError for the unique-together constraint. The DRF serializer should catch this and return HTTP 400 Bad Request. The UI should display a distinct error alert.
Actual Results	IntegrityError caught by the DRF serializer; HTTP 400 returned with body <code>{"non_field_errors": ["Duplicate Mapping Exception: This rule is already mapped."]}</code> ; UI displayed an orange warning toast in 95ms.
Status	PASS
Comments	The constraint is enforced at the database level, making it impossible to create duplicate mappings even via direct API calls.

Table 6.8: TC-08: Wazuh API Handshake and Token Retrieval (Positive)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-08
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the <code>WazuhAPIClient</code> service's capability to establish an authenticated session with the Wazuh Manager REST API by exchanging Basic Auth credentials for a short-lived Bearer token.
Pre-Requisites	1. Wazuh Manager is active at <code>192.168.100.100:55000</code>. 2. Environment variables <code>WAZUH_API_USER</code> and <code>WAZUH_API_PASSWORD</code> are configured.
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Physical Laptop)
Test Scenario	The ICMS backend initializes the <code>WazuhAPIClient</code> and calls the <code>authenticate()</code> method.
Test Steps	1. Trigger <code>WazuhAPIClient.authenticate()</code> from the Django shell. 2. Monitor outbound HTTPS traffic on Port 55000. 3. Confirm the Bearer token is stored on the client instance.
Test Inputs	Host: <code>192.168.100.100</code> , Port: <code>55000</code> , SSL Verify: <code>False</code>
Expected Results	The client should call <code>GET /security/user/authenticate</code> with Basic Auth and store the resulting JWT as <code>self._token</code> .
Actual Results	<code>InsecureRequestWarning</code> suppressed; HTTP 200 received; Bearer token retrieved and stored in 0.8s despite self-signed certificate overhead.
Status	PASS
Comments	Disabling SSL verification is an accepted trade-off in a home-lab deployment; production environments should use a valid CA-signed certificate.

Table 6.9: TC-09: Wazuh Connection Timeout Exception (Negative)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-09
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Simulating a complete network failure during SIEM synchronization to validate that the Celery worker catches the timeout exception and triggers an exponential backoff retry rather than crashing silently.
Pre-Requisites	1. Wazuh Manager taken offline (firewall rule blocks Port 55000). 2. Celery worker configured with <code>max_retries=3</code>.
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Offline)
Test Scenario	A scheduled telemetry synchronization task fires while the SIEM node is unreachable.
Test Steps	1. Block outbound traffic to Port 55000. 2. Execute <code>sync_wazuh_scans.delay()</code>. 3. Monitor Celery logs for exception handling and retry behavior.
Test Inputs	Target Host: 192.168.100.100:55000, Network State: Connection Refused
Expected Results	<code>requests.exceptions.Timeout</code> caught; Celery calls <code>self.retry(countdown=60)</code> to re-enqueue the task.
Actual Results	Timeout raised after 15s; Celery logged "Task retrying in 60s"; task completed successfully on second attempt after connectivity was restored.
Status	PASS (Handled Exception)
Comments	The retry mechanism prevents data loss during transient network failures; the 60-second countdown avoids flooding a recovering SIEM node.

Table 6.10: TC-10: Asynchronous Telemetry Polling via Celery Beat (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-10
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the full end-to-end execution of the <code>sync_wazuh_scans</code> Celery Beat periodic task, from queue dispatch to database commit, confirming the API thread is never blocked during the operation.
Pre-Requisites	1. Redis broker is active. 2. At least 5 agents registered on Wazuh Manager. 3. A periodic task entry exists in the <code>django_celery_beat</code> schedule table.
Environment	Celery Worker / Redis (Docker Ubuntu), Wazuh Manager 4.x
Test Scenario	The scheduled synchronization task is triggered by Celery Beat to poll all 5 registered endpoints.
Test Steps	1. Monitor Celery Beat logs for the dispatch event. 2. Confirm all 5 agents are polled sequentially. 3. Query the <code>ScanResult</code> table for new records after completion.
Test Inputs	Task: <code>sync_wazuh_scans</code> , Agent Count: 5, Trigger: <code>Manual</code>
Expected Results	All 5 endpoints polled; new <code>ScanResult</code> records committed; Django API process remains responsive throughout.
Actual Results	1,200 SCA check rows updated across 5 agents in 12.4s; simultaneous <code>GET /api/dashboard/</code> request responded in 78ms, confirming non-blocking operation.
Status	PASS
Comments	The 78ms concurrent API response confirms the Celery worker tier decoupling is functioning correctly under active load.

Table 6.11: TC-11: Unstructured JSON Log Ingestion (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-11
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Verifying the integrity of the raw log ingestion pipeline when saving a large, semi-structured Wazuh SCA JSON payload into the PostgreSQL <code>jsonb</code> field on the <code>ScanResult</code> model.
Pre-Requisites	1. Wazuh API returning a high-density SCA check payload. 2. PostgreSQL 18 database active and accepting connections.
Environment	Django Backend (Docker Ubuntu), PostgreSQL 18
Test Scenario	A synchronization task receives an 18KB JSON object from the Wazuh API and saves it to <code>raw_log_data</code> .
Test Steps	1. Execute the sync task for a high-density agent. 2. Query the <code>ScanResult</code> table for the <code>raw_log_data</code> field. 3. Compare character count and JSON key structure with the source SIEM payload.
Test Inputs	Payload Source: Wazuh SCA endpoint, Payload Size: 18KB, Field: <code>jsonb</code>
Expected Results	The entire JSON object stored intact with no truncation, and fully traversable via Django ORM for audit retrieval.
Actual Results	18KB payload committed to <code>raw_log_data</code> in 1.1s; character-count comparison confirmed zero data loss; JSON tree fully traversable via ORM.
Status	PASS
Comments	The dual-storage approach (structured boolean + raw JSON) preserves original evidence while enabling fast dashboard queries.

Table 6.12: TC-12: Dynamic Compliance Score Recalculation (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-12
Use Case ID	UC-04
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the scoring aggregation logic by confirming that a new scan result resolving a previously non-compliant control triggers a recalculation of the global compliance percentage, reflected immediately in the frontend gauge.
Pre-Requisites	1. Agent 004 has a baseline score of 75.0%. 2. A failed SCA check for a mapped control exists in the database.
Environment	Celery Worker (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	A manual synchronization resolves a previously failing control, incrementing the global score.
Test Steps	1. Trigger <code>sync_wazuh_scans.delay(agent_id=4)</code> manually. 2. Query <code>DashboardSummaryView</code> for the updated compliance score. 3. Refresh the dashboard and observe the <code>ComplianceGauge</code> animation.
Test Inputs	Agent ID: 004, Control Weight: +6.2%, Initial Score: 75.0%
Expected Results	Global score increments from 75.0% to 81.2%; the React dashboard re-renders the <code>ComplianceGauge</code> SVG with the new value.
Actual Results	Score updated to 81.2% in the database within 2.4s of task completion; gauge SVG re-animated to the new value in 40ms without a page reload.
Status	PASS
Comments	The real-time gauge update is achieved through the Axios polling mechanism in the <code>Dashboard</code> component, re-fetching data every 30 seconds.

Table 6.13: TC-13: Comprehensive PDF Audit Report Generation (Positive)

Module Name	Automated Audit and Reporting Module
Test Case ID	TC-13
Use Case ID	UC-06
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Testing the server-side PDF rendering pipeline's ability to aggregate multi-agent compliance data, apply a Django HTML template, and produce a structurally sound binary PDF with correct page breaks and table alignment.
Pre-Requisites	1. Database contains at least 50 historical <code>ScanResult</code> records across 5 agents. 2. <code>xhtml2pdf</code> is installed and the report HTML template is configured.
Environment	Django Backend (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	An Auditor clicks "Generate Full Report" for the ISO 27001 framework on the Reports page.
Test Steps	1. Authenticate as Auditor and navigate to the Reports page. 2. Select framework "ISO 27001" and click "Generate Report". 3. Download the returned PDF binary and inspect page breaks and table layout.
Test Inputs	Framework: ISO 27001, Date Range: Last 30 Days, Agent Count: 5
Expected Results	A multi-page PDF containing a framework summary, per-agent compliance tables, and a gap analysis section, with clean page breaks and no overlapping elements.
Actual Results	15-page PDF document generated in 4.2s; all inter-section page breaks executed correctly without table overlap; all data values matched the live database query.
Status	PASS
Comments	The 4.2s generation time for a 15-page dossier is acceptable; offloading to the worker tier ensured the auditor's browser session remained interactive throughout.

Table 6.14: TC-14: Asynchronous SMTP Alert Dispatch (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-14
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Validating the automated alert system’s capability to detect a compliance score threshold breach and dispatch a structured HTML email notification via the SMTP relay, without blocking the primary ingestion pipeline.
Pre-Requisites	1. System settings configured with an alert threshold of 80%. 2. SMTP credentials pointing to a MailTrap test inbox are active.
Environment	Celery Worker (Docker Ubuntu), SMTP Relay (MailTrap)
Test Scenario	A telemetry synchronization ingests data dropping Agent 004’s compliance score below the 80% critical threshold, triggering the alert pipeline.
Test Steps	1. Insert a simulated scan result lowering Agent 004’s score to 65.0%. 2. Monitor Celery logs for the <code>send_compliance_alert</code> task dispatch. 3. Check the MailTrap inbox for the received email.
Test Inputs	Agent ID: 004, New Score: 65.0%, Configured Threshold: 80.0%
Expected Results	The scoring module detects the breach, enqueues a <code>send_compliance_alert</code> task, and the worker dispatches an HTML email containing the agent name, current score, and framework details.
Actual Results	Threshold breach detected post-aggregation; <code>send_compliance_alert</code> task dispatched in 1.1s; HTML email received in MailTrap containing Agent 004 metadata, score 65.0%, and a direct link to the agent detail page.
Status	PASS
Comments	The dedicated SMTP worker ensures email dispatch latency does not delay subsequent compliance ingestion tasks queued behind it.

6.4 Non-Functional System Testing

The evaluation of the ICMS platform extended beyond functional correctness to assess the system’s operational constraints, scalability, and overall user experience. This phase of non-functional testing focused on ensuring that the architectural decisions—such as

the separation of concerns and the use of an asynchronous task queue—translate into a performant, secure, and usable application for security analysts.

6.4.1 Performance and Load Testing

The performance evaluation confirmed that the ICMS platform maintains high responsiveness under typical operational loads. Benchmarking revealed that standard API requests, such as retrieving framework lists or mapping configurations, consistently resolve in under 150ms. To handle more intensive I/O operations, such as polling large datasets from the Wazuh SIEM or rendering multi-page PDF reports, the system utilized a Celery worker tier. This architectural choice successfully offloads long-running processes from the main request-response cycle, preventing the Nginx and Django gateway from timing out during heavy stress. Even when processing 50 simultaneous synchronization tasks, the system remained stable, completing all operations without degrading the user interface performance.

Table 6.15: Performance Test Results

Test Scenario	Metric Measured	Result	Status
React SPA cold start to Login	Screen load time	Under 1.5 seconds	Pass
Django API framework list fetch	Response latency	Under 150 ms	Pass
Wazuh SIEM Sync (50 agents)	Celery execution time	Under 15 seconds	Pass
Compliance gauge re-render	UI update latency	Under 50 ms	Pass
ISO 27001 PDF generation (15 pages)	Server rendering time	Under 4.5 seconds	Pass
SMTP alert email dispatch	Queue delivery time	Under 1.5 seconds	Pass

6.4.2 Security and Penetration Testing

A defense-in-depth strategy was implemented to secure the platform against unauthorized access and data manipulation. The system relies on the Django ORM to provide inherent protection against SQL injection attacks by utilizing parameterized queries for all database interactions. Identity management is handled through a stateless JWT ar-

chitecture, utilizing short-lived access tokens and rotating refresh tokens to minimize the window for potential session hijacking. Additionally, the integration of TOTP-based 2FA provides a cryptographically secure second layer of authentication, ensuring that administrative access remains protected even in the event of primary credential compromise.

Table 6.16: Security Test Results

Security Control	Test Description	Status
JWT Session Enforcement	Attempted access to protected Dashboard without token; React Router successfully redirected to login.	Pass
Role-Based Access Control	Attempted to POST mapping rules using an Auditor token; DRF returned HTTP 403 Forbidden.	Pass
SQL Injection Prevention	Submitted malicious SQL payloads in search fields; Django ORM parameterized queries safely.	Pass
TOTP 2FA Verification	Attempted to bypass the 2FA prompt after password entry; backend withheld the final access token.	Pass
XSS Sanitization	Injected HTML scripts into framework descriptions; React DOM safely escaped the output.	Pass

6.4.3 Usability and Accessibility Testing

The user interface was designed to minimize cognitive load and reduce analyst fatigue in high-pressure Security Operations Center (SOC) environments. Central to this is the implementation of a global `ThemeContext`, which allows toggling between Light and Dark modes. This gives analysts control over the UI palette based on local lighting conditions, reducing eye strain during long shifts. Furthermore, the platform organizes complex compliance data into paginated and filterable tables, ensuring that large datasets remain manageable and readable without overwhelming the user's screen.

Table 6.17: Usability Test Task Completion Results

Task	Avg. Completion Time	Success Rate	Status
Log in and complete 2FA verification	0.8 minutes	5 / 5 (100%)	Pass
Toggle Light/Dark mode context	0.2 minutes	5 / 5 (100%)	Pass
Map a Wazuh Rule to an ISO Control	1.5 minutes	5 / 5 (100%)	Pass
Trigger a manual endpoint scan	1.0 minutes	5 / 5 (100%)	Pass
Generate and download PDF report	1.2 minutes	4 / 5 (80%)	Pass

6.4.4 Compatibility and Deployment Testing

The ICMS platform underwent compatibility testing to ensure a consistent experience across modern web browsers, including Chrome, Firefox, and Edge. The React SPA maintained its layout integrity and functionality across these environments, confirming the reliability of the frontend codebase. Deployment testing validated the containerized architecture by performing multiple teardowns and spin-ups using Docker Compose. By encapsulating the entire stack—including Django, React, Redis, Celery, and PostgreSQL—the deployment remains deterministic across different host environments. This containerized approach ensures that the platform can be reliably deployed on any Ubuntu-based machine without encountering dependency conflicts.

Table 6.18: Compatibility and Deployment Test Results

Environment / Scenario	Test Description	Status
Cross-Browser UI Rendering	Executed React SPA on Chrome, Firefox, and Edge; Flexbox layouts and charts rendered correctly.	Pass
Docker Compose Teardown	Executed <code>docker-compose down -v</code> and rebuilt stack; all 7 containers initialized with Exit Code 0.	Pass
Database Volume Persistence	Restarted PostgreSQL container; verified that previous user accounts and compliance data remained intact.	Pass
Celery Broker Recovery	Temporarily stopped Redis container; verified Celery gracefully resumed task queuing upon Redis restart.	Pass

Chapter 7

Conclusions

The development and deployment of the Intelligent Compliance Management System (ICMS) successfully demonstrated that integrating raw Security Information and Event Management (SIEM) telemetry directly into a centralized Governance, Risk, and Compliance (GRC) engine can automate the traditionally manual auditing lifecycle. Originating as an industrial collaborative project, the platform bridged the gap between theoretical regulatory frameworks and practical endpoint security by engineering a deterministic mapping algorithm. This implementation proved that by translating Wazuh Security Configuration Assessment (SCA) alerts into structured ISO 27001 controls, an organization can maintain a continuous, mathematically verifiable compliance posture rather than relying on point-in-time assessments.

Several critical engineering conclusions were drawn from the architectural implementation. First, the integration of a hybrid database schema—utilizing PostgreSQL’s `JSONField` for unstructured Wazuh logs alongside strict relational tables for user roles and frameworks—provides an efficient mechanism for ingesting unpredictable SIEM data without compromising the ACID compliance of the core application. Second, the evaluation phase confirmed that offloading heavy I/O-bound operations, such as API polling and multi-page PDF generation, to a distributed Celery worker queue is a mandatory architectural pattern to prevent API gateway timeouts in high-density enterprise environments. Finally, the deployment demonstrated that stateless JSON Web Tokens (JWT) combined with a React Context provider offers an efficient mechanism for enforcing strict Role-Based Access Control (RBAC) across a Single Page Application without incurring server-side session management overhead.

Moving forward, there are several avenues through which this industrial project can be expanded into a polished, enterprise-ready product. A primary technical enhancement would involve transitioning the SIEM integration from periodic REST API polling to a persistent WebSocket or Webhook-driven streaming architecture, which would eliminate latency and enable true real-time compliance state recalculations. Additionally, the system’s containerized topology can be upgraded from standard Docker Compose to Kubernetes manifests to support horizontal scaling and high availability across distributed

cloud clusters. The long-term vision for the ICMS platform is to refine its core orchestration engine and release it as an open-source software solution. By open-sourcing the codebase, the platform can foster community-driven development, allowing external engineers to contribute integration modules for additional SIEM providers, such as Splunk or Elastic Security, and continuously expand the library of supported global regulatory frameworks.

Appendix A

User Manual

A.1 Purpose

The Intelligent Compliance Management System (ICMS) is developed as a centralized Governance, Risk, and Compliance (GRC) platform. The system aims to eliminate manual spreadsheet auditing, reduce operational overhead, and provide a secure, automated environment for mapping regulatory frameworks to technical endpoint telemetry ingested from the Wazuh SIEM.

A.2 System Requirements

The following requirements are necessary for the smooth operation of the system:

- **Device:** Desktop or workstation computer (Minimum 1080p resolution recommended for data tables).
- **Web Browser:** Modern web browser (Google Chrome v100+, Mozilla Firefox v100+, or Microsoft Edge).
- **Network:** Secure intranet or VPN access to the ICMS hosting environment.
- **Authentication:** A registered Authenticator Application (e.g., Google Authenticator, Authy) for TOTP Two-Factor Authentication.

A.3 System Access

- Open a supported web browser and navigate to the ICMS platform URL provided by your system administrator.
- Enter your registered corporate email and password.

- Upon successful credential validation, enter the 6-digit TOTP code from your authenticator application.
- After successful login, the system will redirect you to the appropriate Dashboard based on your assigned Role-Based Access Control (RBAC) profile (Superadmin, Admin, or Auditor).

A.4 Main User Steps

Step 1 – Authentication and Role Verification

- Users authenticate via the primary login portal.
- The system verifies the JWT token and mounts the interface corresponding to the user's privilege level.

Step 2 – Dashboard Overview

- Navigate to the Home Dashboard to view the global organizational compliance posture.
- View dynamic gauges representing real-time compliance percentages across all active frameworks.
- Toggle between Light Mode and Dark Mode using the theme switcher in the navigation bar to reduce eye strain.

Step 3 – Framework and Control Management (Admins)

- Navigate to the "Frameworks" tab to view active regulatory standards (e.g., ISO 27001).
- Select a framework to drill down into its specific clauses and technical controls.

Step 4 – Telemetry Mapping Configuration (Superadmins)

- Navigate to the "Mapping Configuration" portal.
- Link specific Wazuh Security Configuration Assessment (SCA) Rule IDs to the appropriate GRC controls.

- Save the mapping to instruct the engine on how to categorize incoming SIEM events.

Step 5 – Wazuh Telemetry Synchronization

- The system automatically polls the Wazuh manager in the background.
- To force an immediate update, Administrators can click "Trigger Manual Sync" on the Scans page.
- View the synchronization logs to verify successful data ingestion.

Step 6 – Endpoint Monitoring

- Navigate to the "Endpoints" or "Agents" view.
- Select a specific workstation or server to view its localized compliance score and a list of failed security checks.

Step 7 – Automated Audit Report Generation

- Navigate to the "Reports" module.
- Select the desired regulatory framework and date range.
- Click "Generate PDF". The system will process the data and prompt a download of the immutable audit dossier.

Step 8 – System and User Management

- Superadmins can navigate to "Manage Users" to provision new accounts and assign RBAC roles.
- Configure SMTP threshold alerts in the System Settings to ensure immediate email dispatch upon compliance failure.

A.5 Logout

- Click the user profile avatar in the top right corner of the navigation shell.
- Select "Logout" to securely invalidate the active JWT session and clear browser storage.

A.6 Support

- For credential recovery or 2FA resets, contact the internal IT Security Administrator.
- For Wazuh agent connectivity issues, consult the SIEM infrastructure team.

A.7 Note

The ICMS platform is designed to automate compliance visibility and reporting. While the system ensures mathematically deterministic mappings based on SIEM telemetry, it is an orchestration and governance tool. Users and administrators are advised that maintaining compliance requires active remediation of the vulnerabilities identified by the platform; the ICMS highlights the security gaps, but direct administrative action on the endpoints is required to resolve them.

Bibliography

- [1] Center for Internet Security. Cis docker benchmark v1.2.0. Technical report, 2020. URL <https://www.cisecurity.org/benchmark/docker>.
- [2] ISO/IEC. Information security, cybersecurity and privacy protection — information security management systems — requirements, 2022. URL <https://www.iso.org/standard/27001>.
- [3] Arnold Johnson, Mark Kelley, and Barrett Ronquille. Guide for security-focused configuration management of information systems. Technical Report NIST Special Publication 800-128, National Institute of Standards and Technology, 2011.
- [4] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Technical Report NIST Special Publication 800-92, National Institute of Standards and Technology, 2006.
- [5] Ivanko Kuraj. *Microservices Architecture for Heterogeneous-Database Web Applications*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, 2024. URL <https://dspace.mit.edu/handle/1721.1/153838>.
- [6] Mozilla Developer Network. Basic concepts of flexbox, 2024. URL https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox. Accessed: 2026-04-23.
- [7] D. M’Raihi, S. Machani, M. Pei, and J. Rydell. Totp: Time-based one-time password algorithm. RFC 6238, Internet Engineering Task Force, 2011.
- [8] National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations*. NIST Special Publication 800-53 Revision 5. U.S. Department of Commerce, 2020. doi: 10.6028/NIST.SP.800-53r5.
- [9] J. Nelson. Securing the stateless frontier: A critical analysis of advanced authentication techniques in asp.net core web apis. *ResearchGate*, 2024. Technical Analysis of JWT and RBAC.
- [10] Massachusetts Institute of Technology. The psychology of alert fatigue in security operations. *MIT Sloan Management Review*, 2021. Technical Report on HCI and Security.

- [11] PlanetScale. What is the n+1 query problem?, 2023. URL <https://planetscale.com/blog/what-is-the-n-1-query-problem-and-how-to-fix-it>. Accessed: 2026-04-23.
- [12] Murugiah Souppaya, John Morello, and Karen Scarfone. Application container security guide. Technical Report NIST Special Publication 800-190, National Institute of Standards and Technology, 2017.
- [13] U.S. Congress. Federal information security modernization act of 2014, 2014. Public Law 113-283.
- [14] Wazuh Team. Wazuh documentation, 2024. URL <https://documentation.wazuh.com/>. Accessed: 2026-04-23.
- [15] T. Zavarella. *The impact of alert fatigue on cognitive load in security operations*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, 2022. URL <https://dspace.mit.edu/handle/1721.1/155176>.